

תכנון ספרתי מתקדם – סמסטר א תשפ"ד

סיכום חלק ההרצאות¹

המרכז האקדמי לב, ירושלים

תואר ראשון בהנדסת חשמל ואלקטרוניקה

מרצה: מר אורי שטרו

סיכום: אליעזר טויק

הסיכום הוא חומר עזר בלבד. החומר המחייב הוא החומר המועבר בשיעור, ובנוסף, המעבדות כולן עם פתרונותיהן.

תוכן עניינים

שבוע 1	2
מבנה הקורס, הציון ושאר דברים טכניים	2
מבוא לשפת VHDL	2
כללי כתיבה לשפת VHDL	3
דוגמאות ל-entity של רכיבים שונים	4
שבוע 2	5
חובת הצהרת ספריות ב-VHDL	5
קבצי התכנון	6
ATP ו-Spec	6
Verification & Validation (V&V)	6
תהליך ההוצאה מן הכח אל הפועל של תכנון ספרתי	6
ארכיטקטורה ב-VHDL	7
שבוע 3	7
ארכיטקטורה באמצעות components	7
sensitivity list בתחילת ה-process מול שימוש ב-wait	8
after	9
שימוש ב-wait ל'תקיעת' process	9
הסינתזה	10
תופעת ה-glitch בלוגיקה קומבינטורית	10
Flip-Flop מול Latch	11
שבוע 4	12
RESET	12
Clock enable	12
RTL coding style	13
כתיבת קוד VHDL לתכנון Flip-Flop ללא שימוש ב-wait	13
שבוע 5	13
הסינתזה	13
סוגי משתנים ב-VHDL	13
שבוע 6	16
המשך נושא סוגי משתנים ב-VHDL	16
סדרי קדימויות בפעולות לוגיות ומתמטיות ב-VHDL	17

¹ לקורס יש חלק נוסף – מעבדות מעשיות, והוא חלק אינטגרלי מחומר הקורס, שלא כלול במלואו בסיכום זה.

17	פעולות בין סוגים שונים של משתנים או ערכים.....
18	איחוד משתנים או מערכים למערך אחת (concatenation).....
18	הביטוי <=.....
18	שתי צורות של כתיבת קוד ב-VHDL.....
18	השימוש בתנאי if, else ו-elsif.....
19	השימוש ב-case.....
19	השוואה בין שימוש ב-case לבין שימוש ב-if,else.....
20	שימוש בתנאי if ללא else או שימוש ב-case ללא התייחסות לכל האפשרויות.....
20	כתיבה מקוצרת ללא המעטפת המלאה של process.....
21	דוגמאות למבנים נפוצים של לוגיקה קומבינטורית.....
24	שבוע 7.....
24	דוגמאות נוספות של מבני לוגיקה קומבינטורית נפוצים.....
26	היחס לסתירה בין שני אותות כאשר משתמשים ב-std_logic או ב-std_ulogic.....
27	פין שיכול לשמש ככניסה או כציאה.....
28	שבוע 8.....
28	שלשה סוגי testbench.....
28	שערים לוגיים באמצעות טכנולוגיית CMOS.....
32	שיקולים חשמליים בבחירת שערים לוגיים למעגל.....
33	שבוע 9.....
33	שערים לוגיים AOI ו-OAI.....
39	מימוש MUX באמצעות Transmission Gate.....
41	טכנולוגיות למימוש מעגלים גדולים על סיליקון – פתיחה.....
41	טכנולוגיית Full Costume.....
41	טכנולוגיית Gate Array.....
41	שבוע 10.....
42	המשך טכנולוגיית Gate Array.....
43	טכנולוגיית FPGA.....
44	בחירת רכיב FPGA עבור מימוש התכנון.....
45	שיעורי השלמה.....
45	מידע מעשי עבור שימוש בטכנולוגיית FPGA.....
45	טכנולוגיית Structured ASIC.....
45	טכנולוגיית CPLD.....
48	טכנולוגיית Standard Cell.....

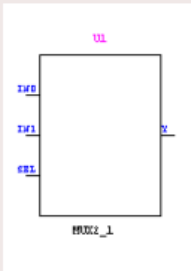
שבוע 1

מבנה הקורס, הציון ושאר דברים טכניים

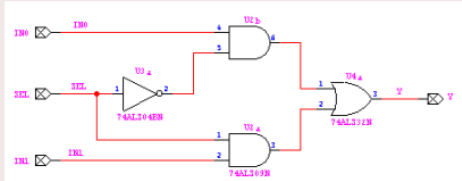
בהרצאה הוצגה המצגת "הקורס: הסבר, מהות דרישות ונהלים" (מופיעה במודל כמעט בתחילת הדף של הקורס). יש במצגת את המידע הרלוונטי כולל התאמות לימי המלחמה. יש במצגת המלצה לספרים עם חומר שיוכל לסייע לסטודנטים. אין חובה לעיין בהם.

מבוא לשפת VHDL

עברנו על המצגת "פרק מבוא מקוצר לשפת VHDL", שמופיעה תחת הכותרת "מפגש 1". במצגת ראינו בין השאר את השקף הבא, שמשווה בין שרטוט של רכיב (ללא התוכן הפנימי שלו), לבין התיאור של הרכיב בשפת VHDL:

Schematic Capture	VHDL
Symbol	Entity
	<pre> entity MUX2_1 is port (IN0: in std_logic; IN1: in std_logic; SEL: in std_logic; Y : out std_logic); end MUX2_1; </pre>

אחרי תיאור כזה (שנקרא entity) בשפת VHDL, יגיע תיאור של ה'קריבים' של הרכיב (שנקרא architecture). בשקף הבא מובאת השוואה בין שרטוט של הלוגיקה הפנימית של רכיב לבין תיאור שלו בשפת VHDL:

Schematic Capture	VHDL
Schematics	Architecture
	<pre> architecture MUX2_1_ARC of MUX2_1 is Begin process(SEL, IN0, IN1) begin if (SEL = '0') then Y <= IN0; else Y <= IN1; end if; end process; end MUX2_1_ARC; </pre>

דרך אגב, בשפת תיאור תוכנה שנקראת Verilog (ואותה לומדים בקורס בחירה למעוניינים), התיאור של הרכיב הזה נראה כך:

```

////////////////////////////////////
// mux2_1.v
//
// 2:1 multiplexer
////////////////////////////////////
module mux2_1 (y, in0, in1, sel);
  output y;
  input in0, in1, sel;
  reg y;

  always @(in0 or in1 or sel)
  begin
    if (!sel)
      y = in0;
    else
      y = in1;
    end
  end
endmodule
////////////////////////////////////

```

כפי שרואים, בשפת Verilog אין חלוקה ל-entity ול-architecture. שתי השפות, גם VHDL וגם Verilog, קיימות בתעשייה.

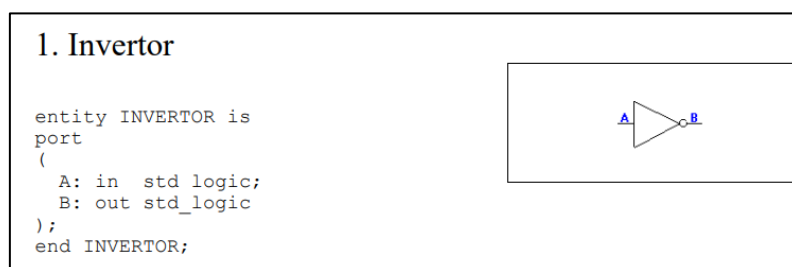
כללי כתיבה לשפת VHDL

- בשפת VHDL אין הבדל בין אותיות גדולות או קטנות (בשונה מ-Verilog). כלומר המילה name שווה בדיוק למילה NAME או למילה NaMe.
- אפשר להשתמש רק באותיות או מספרים.

- שם של רכיב או סיגנל (בהמשך נלמד מה זה סיגנל) חייב להתחיל באות ולא במספר.
- שם של רכיב או סיגנל אינו יכול להכיל רווח. כדי ליצור רווח יש להשתמש ב ' _ '.
- התו ' _ ' אינו יכול להופיע בתחילה או בסוף של שם, וגם אינו יכול להופיע פעמיים רצוף.
- יש מילים שמורות (מופיעות בקובץ תחת הכותרת 'מפגש 1' במודל), שבהן אין להשתמש מעבר לשימוש הקבוע שלהן.
- בסוף מבנה יופי התו ' ; '.
- כל הגדרה או הצהרה (שורה) בתוך המבנה מסתיימת בתו ' ; ' , חוץ מהאחרונה באותו מבנה.

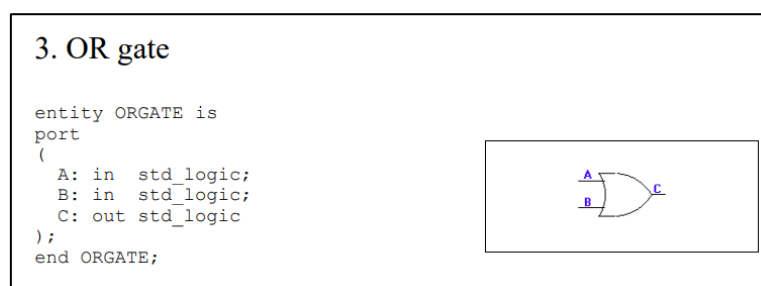
דוגמאות ל- entity של רכיבים שונים

רכיב מהפך (שער 'NOT') :

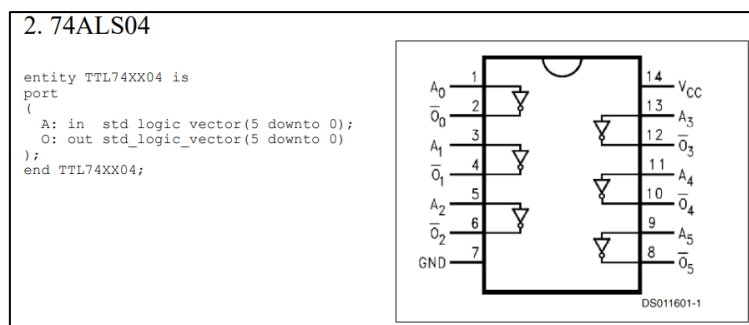


הסבר :

- בתחילת כל שורה כותבים את שם הכניסה/יציאה, אחרי נקודתיים, ואחר כך את הסוג שלה.
- in std logic מגדיר כניסה לוגית. כניסה כזו יכולה לקבל 0 או 1 – שבפועל יתבטאו בשתי רמות מתחים אפשריות, שכל אחת מהם תייצג אחד מהערכים 1 או 0.
- out std logic מגדיר יציאה לוגית.
- יש לציין, שלא מתואר כאן בכלל מה הרכיב עושה, אלא רק מה הם הכניסות והיציאות שלו. זה כל מה שאמור להכיל entity.
- דוגמה נוספת, של שער OR :



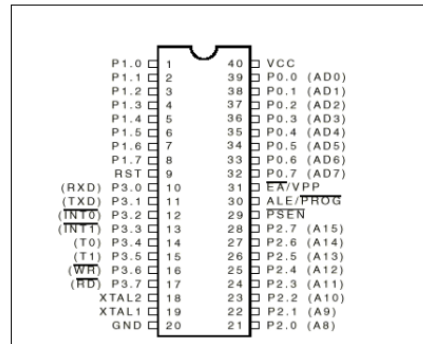
דוגמה נוספת, של רכיב שיש לו סדרת כניסות וסדרת יציאות :



הביטוי std logic vector מגדיר סדרה של כניסות או יציאות. המספור של הכניסות או היציאות בסדרה מיוצג בביטוי 5 downto 0, כלומר מ-5 עד ל-0 בספירה לאחור.

5. 8051 microcontroller

```
entity MCS51 is
port
(
  P0:    inout std_logic_vector(7 downto 0);
  P1:    inout std_logic_vector(7 downto 0);
  P2:    inout std_logic_vector(7 downto 0);
  P3:    inout std_logic_vector(7 downto 0);
  RESET: in    std_logic;
  XTAL1: in    std_logic;
  XTAL2: out   std_logic;
  ALE:    out   std_logic;
  PSEN_N: out   std_logic;
  EA_N:   in    std_logic
);
end MCS51;
```



הקו מעל שמות חלק מהפינים, מציין שהם active low, כלומר דווקא אם יוזן להם הערך 0, הוא הערך שיגרום להם לבצע פעולה, והערך 1 ימנע מהם לבצע אותה. כניסה זו היא כאילו 'הפוכה', ומקובל לסמנה בקו עליון. מומלץ לקרוא לכניסות אלו בשם שמציין זאת, בדוגמה שכאן נוספה האות N לשמות של הכניסות הללו.

הביטוי inout שמופיע בקוד, פירושו פין שיכול לשמש ככניסה או כיציאה, ואינו מוגדר מראש בתכנון דווקא לאחד התפקידים.

בשתי הדוגמאות האחרונות שראינו, אין בקוד ה-VHDL הצהרה על שני הפינים GND (אדמה - ground) ו-VCC (המתח העליון של המעגל). פינים אלו אינם חלק מהפונקציות הלוגיות של רכיבים דיגיטליים, אלא משמשים עבור אספקת מתח ההפעלה לחלקי המעגל שבתוך הרכיבים.

שבוע 2

חובת הצהרת ספרייה ב-VHDL

בכתיבת קוד ב-VHDL אנחנו משתמשים במושג std_logic ובמושגים נוספים שההגדרה שלהם קבועה בתקן IEEE_1164. כאשר משתמשים בהם מתוך הסתמכות על התקן הזה, צריך להצהיר בתחילת הקוד על השימוש בספרייה הרלוונטית. כך כלי ה-VHDL יבינו את משמעותם של מושגים אלו. אחר ההצהרה תבוא גם פקודת שימוש בספרייה, שאומרת לכלי ה-VHDL להשתמש במה שרלוונטי לענייננו מתוך אותה ספרייה. ההצהרה ופקודת השימוש נראות כך :

```
library IEEE;
use IEEE.std_logic_1164.all;
```

בשורת ההצהרה אנו מצהירים על הספרייה כולה. בפקודת השימוש אנו אומרים לכלי ה-VHDL להשתמש בכל מה שקיים ב-package² שנקרא std_logic_1164. פקודת השימוש בנויה 'מלמעלה למטה', כלומר מתייחסת לספרייה הגדולה IEEE, אחר כך ל-package בתוך הספרייה הזו, שנקראת std_logic_1164 ואחר כך מתייחסת לכל מה שכלול באותה package. עקרונית היה אפשר לבחור דברים ספציפיים מתוך ה-package, אבל יותר נח פשוט לבחור הכל.

בהמשך נכיר יותר את התקן של std_logic, אבל כרגע, מבחינתנו std_logic הוא ביט שיכול להכיל 0 או 1, ו-std_logic_vector הוא סדרה של ביטים כאלה.

יש ספרייה נוספת, בשם work, שעליה אין צורך להצהיר. ספרייה זו היא הספרייה (תיקיה) הכוללת את התכנון שלנו בתוכה.

² בעתיד נלמד כנראה על משמעות המושג.

[במסגרת מטלה 1 אין צורך להצהיר על הספרייה ולכתוב את פקודת השימוש.]

קבצי התכנון

הקבצים שבהם נכתוב את התכנון ב-VHDL יהיו קבצים עם סיומת vhd. נערוך קבצים אלו באמצעות תוכנת ++notepad, שקיימת במחשבי המעבדה (ובמודל יש גם הוראות להורדת תוכנות נוספות שרגישות לשפת VHDL). כשפותחים קובץ כזה ב-windows מומלץ ללחוץ עליו לחיצה ימנית ולבחור פתיחה באמצעות ++notepad, כי לפעמים windows עלול שלא להבין מהו קובץ עם סיומת כזו.

Spec ו-ATP

Spec (קיצור של specification) הוא מסמך כלשהו שמתאר מוצר מסוים. הוא יכול לכלול פרמטרים מסוגים שונים, לפי סוג המוצר.

ATP הוא מסמך המכיל סדרת בדיקות עבור מוצר מסוים. לדוגמה: עבור מהפך (שער NOT) אפשר להגדיר: הכנס 1 ובדוק ביציאה האם יש 0. אחר כך הכנס 0, ובדוק אם ביציאה יש 1.

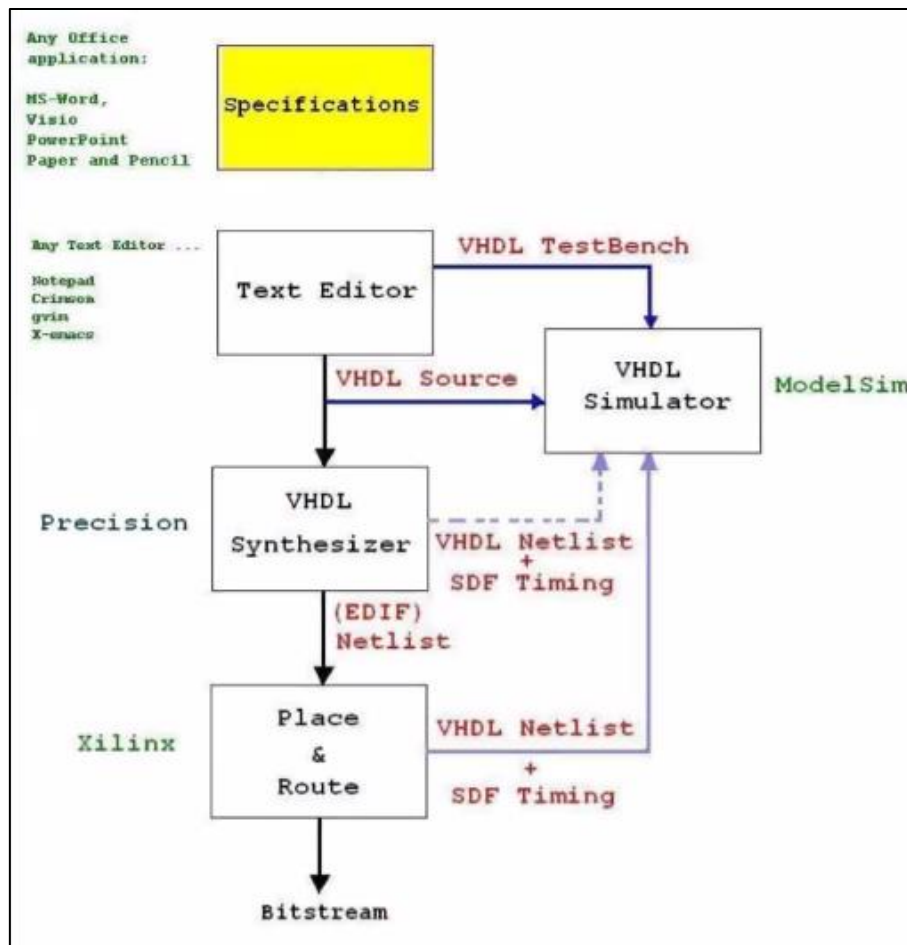
Verification & Validation (V&V)

Verification (ורפיקציה – אימות) בודקת האם המוצר תואם לתכנון שלו (בהתאם ל-spec). Validation (ולידציה – תיקוף) היא בדיקה האם המוצר ממלא את ייעודו.

נסביר את ההבדל באמצעות דוגמה: במסמך spec מתוארת חולצה. אחרי הייצור נבדוק בשלב האימות שהחולצה מתאימה ל-spec. בשלב התיקוף נבדוק שהחולצה אכן מתאימה לשימוש ולמקומות שאליהם מעוניין הלוש להגיע כשהיא עליו.

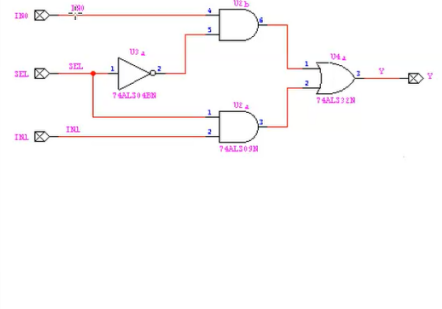
תהליך ההוצאה מן הכח אל הפועל של תכנון ספרתי

תרשים הזרימה הבא מתאר את התהליך מהרעיון הראשוני ועד המוצר המוגמר. כל חלק מהקורס הזה (והקורס הבא) יעסוק לפחות באחד מהנושאים המופיעים בתרשים.



ארכיטקטורה ב-VHDL

בשיעור הקודם למדנו שבארכיטקטורה (architecture) מופיע אופן הפעולה של הרכיב שמתכננים. נכיר כעת שתי דרכים לכתוב ארכיטקטורה. נתבונן בדוגמה הבאה, שמתארת MUX³ בתרשים מול הארכיטקטורה שלו:

Schematic capture	VHDL
schematics	architecture
	<pre>architecture MUX2_1_ARC of MUX2_1 is begin A_LABEL: process(SEL, IN0, IN1) begin if (SEL = '0') then Y <= IN0; else Y <= IN1; end if; end process; end MUX2_1_ARC;</pre>

הארכיטקטורה כאן נכתבה בדרך שנקראת behavioral (התנהגותי), ומתארת מה ה-MUX צריך לעשות במצבים שונים.

דרך נוספת לכתיבת ארכיטקטורה היא dataflow, המתארת את הקשר בין הכניסות ליציאות באמצעות משוואה. את ה-MUX אפשר לתאר בשיטה זו כך:

```
architecture DATAFLOW_ARC of MUX2_1 is
begin
  Y <= (IN0 and not(SEL)) or (SEL and IN1);
end DATAFLOW_ARC;
```

בכל ארכיטקטורה יש להקפיד על הכללים הבאים:

- בשורה הראשונה: architecture, אחר כך השם של הארכיטקטורה, המילה of, השם של הרכיב שעבורו מיועדת הארכיטקטורה (לפי השם שנתנו לו ב-entity) ולבסוף המילה is.
- בסוף הארכיטקטורה תופיע המילה end, לאחריה שם הארכיטקטורה (לפי השם שנכתב בשורה הראשונה) ולבסוף נקודה פסיק.
- כדי להכניס ערך ממיקום אחד לשני משתמשים ב <= (שמחליף את הסימן =).
- בארכיטקטורה מסוג behavioral הפעולות שמונתות ב-if ו-else צריכות להיות בתוך process. שבתחילתו תופיע המילה process (לפעמים תופיע לאחריה באותה שורה בסוגריים כל הכניסות הרלוונטיות לאותו process) ובסוף ה-process יופיע המשפט end process עם נקודה פסיק סמוך לו.

הערה: ישנן דרכים נוספות לכתיבת ארכיטקטורה.

שבוע 3

ארכיטקטורה באמצעות components

בשבוע שעבר הכרנו שתי שיטות לכתיבת ארכיטקטורה, וראינו דוגמאות של תכנון MUX באמצעות שתי השיטות. ישנה שיטה נוספת: באמצעות הגדרת components, כלומר רכיבים חלקיים, שכולם יחד ירכיבו את הרכיב הרצוי.

³ רכיב שתפקידו להוציא החוצה את המידע שבאחת מכניסותיו. בחירת הכניסה שהמידע שלה יוצג ביציאה נעשה באמצעות הכנסת הערך המתאים בכניסה select (נקראת SEL בדוגמה שלנו).

בדוגמה של ה-MUX, אפשר לקחת את השערים הלוגיים שמופיעים בתרשים שראינו בשבוע שעבר, ולהרכיב מהם את ה-MUX. כאן מופיע קוד של ארכיטקטורה כזו:

<pre>library IEEE; use IEEE.std_logic_1164.all; entity MUX2_1 is port (IN0: in std_logic; IN1: in std_logic; SEL: in std_logic; Y: out std_logic); end MUX2_1; architecture STRUCTURAL_ARC of MUX2_1 is component INV1 port (A: in std_logic; B: out std_logic); end component; component AND2 port (A, B: in std_logic;</pre>	<pre> C: out std_logic); end component; component OR2 port (A, B: in std_logic; C: out std_logic); end component; signal INV_SEL: std_logic; signal X0, X1: std_logic; begin U0: INV1 port map (SEL, INV_SEL); U1: AND2 port map (IN0, INV_SEL, X0); U2: AND2 port map (IN1, SEL, X1); U3: OR2 port map (X0, X1, Y); end STRUCTURAL_ARC;</pre>
---	---

(לא נרחיב כעת על הנושא, כי הוא יילמד בהמשך).

sensitivity list בתחילת ה-process מול שימוש ב-wait

כפי שלמדנו בשבוע הקודם, בארכיטקטורה מסוג behavioral משתמשים ב-processes (כשמדברים בעברית אפשר לומר 'פרוססים'). ה-process הוא המבנה הכי בסיסי בשפת תכנון חומרה. ה-process מתבצע שוב ושוב בהתאם למאורעות שונים. אפשרות אחת לציין את המאורעות שיגרמו ל-process להתרחש היא באמצעות רשימת רגישויות (sensitivity list). רשימה כזו תבוא בסוגריים לאחר המילה process שבתחילת ה-process. ברשימה זו יופיעו אותות הכניסה הרלוונטיים ל-process. בכל פעם שיארע שינוי באחד מהם, ה-process יקרה מחדש. אם לא מגדירים אות מסוים ברשימת הרגישויות, האות לא ישפיע כלל על ה-process, אף אם הוא יופיע בתוך ה-process. אפשרות אחרת לציין אותות כניסה שיגרמו ל-process כאשר הם ישתנו היא באמצעות wait. נבין את השימוש בו באמצעות הדוגמה הבאה:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity DFF is
port(
    CLK: in std_logic;
    D: in std_logic;
    Q: out std_logic
);
end DFF;
architecture DFF_ARC of DFF is
begin
    THE_FLIP_FLOP: process
    begin
        wait until rising_edge(CLK);
        Q <= D;
    end process;
end DFF_ARC;
```

מדובר על Flip-flop מסוג D, שבכל עליית שעון מעביר את הערך הלוגי שנמצא ב-D ל-Q. בקוד שמופיע כאן, הפקודה שמעבירה את המידע מ-D ל-Q נכתבה אחרי הביטוי wait until rising edge (CLK), ולכן היא תתרחש בכל עליית שעון. יש כמה סוגים של wait. נכיר כאן שלושה מביניהם:

- כאשר אנחנו כותבים 'wait until rising edge (CLK)', הכוונה היא שהפעולה הבאה תתבצע כאשר תתרחש עליית שעון. הפעולה תתבצע שוב ושוב בכל פעם שתתרחש עליית שעון.

- כאשר אנחנו כותבים 'wait for', הכוונה היא שהפעולה הבאה תתבצע אחרי שיעבור פרק זמן מסוים שאנחנו מגדירים. הפעולה תתבצע שוב ושוב בהפרש הזמן שקבענו.
- הפקודה 'wait on' מקבילה לרשימת רגישויות, והפעולה שמופיעה אחריה תתרחש כאשר יארע שינוי באחד האותות המוזכרים אחרי המילים wait on.

הדגשה: האופציה של רשימת רגישויות והאופציה של wait לא באות יחד! כאשר מופיע בתחילת process רשימת רגישויות, אסור שבתוך ה-process יופיע wait.

after

יש אפשרות נוספת לכתיבה מקוצרת של process, בצורה שתכלול גם השהייה, ללא ציון רשימת רגישויות או wait, על ידי כתיבת הביטוי after. נראה כאן דוגמה:

```
architecture INV_ARC of INV is
begin
  THE_B: B <= not(A) after 5 ns;
end INV_ARC;
```

מדובר על מהפך (שער NOT), שביציאתו יופיע ערך לוגי הפוך מהערך שבכניסתו, אך בשונה ממהפך רגיל, שבו ההשפעה של הכניסה על היציאה היא (כמעט) מיידיית, כאן תהיה השהייה של 5 ננו-שניות בין שינוי ערך הכניסה לבין שינוי ערך היציאה.

בדוגמה הבאה מופיעה אפשרות נוספת לשימוש ב-after, במהלך סימולציה (למשל ב-Modelsim), כאשר מעוניינים לשנות ערך של אות מסוים למשך פרקי זמן קבועים מראש:

```
architecture WFM_ARC of WFM is
signal WFM_SIG: std_logic;
begin
  THE_WFM_SIG: WFM_SIG <= '0',
                                '1' after 100 ns,
                                '0' after 250 ns,
                                '1' after 300 ns,
                                '0' after 700 ns,
                                '1' after 850 ns;
end WFM_ARC;
```

המשתנה SIG_WFM (קיצור של waveform signal) ישתנה בין 1 ל-0 לפי ההשהיות שנכתבו כאן. כאשר משתמשים ב-after, הזמנים הם יחסית לראשית, ולא יחסית לפקודה הקודמת. כלומר: בתחילה האות יהיה 0, לאחר 100 ננו-שניות הוא יהפוך ל-1, לאחר 150 ננו-שניות נוספות (שהן 250 ננו-שניות מתחילת התהליך) הוא יהפוך ל-0, לאחר 50 ננו-שניות נוספות (שהן 300 ננו-שניות מתחילת התהליך) הוא יהפוך ל-1, וכן הלאה.

שימוש ב-wait ל'תקיעת' process

כפי שאמרנו, כרגיל process חוזר על עצמו בלי סוף, בתנאי שיתרחש אירוע כלשהו בכל פעם. אך יש אפשרות לבצע process חד פעמי שלא יחזור על עצמו, באמצעות שילוב המילה wait בסופו. מצורפת כאן דוגמה:

```
architecture EXAMPLE of BARE_WAIT is
signal SIG: std_logic;
begin

  P0: process
  begin
    SIG <= '1';
    wait;
  end process;

end EXAMPLE;
```

דרך אגב: הסיגנל (אות) SIG הוא סיגנל פנימי בתוך הרכיב, ואיננו כניסה או יציאה שהופיעו ב-entity. לכן היה צריך להצהיר עליו בשורה: signal SIG: std_logic;

הסינתזה

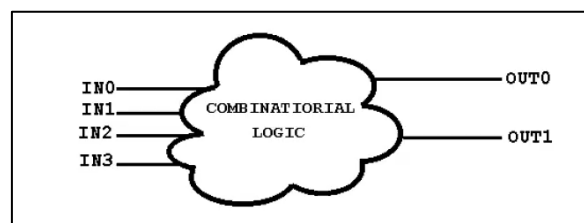
עד כאן עסקנו בכתיבת קוד VHDL ובמעבדות ביצענו אימון על כלי הסימולציה – שמדמה את הכניסות והיציאות של הרכיב שאת תיאורו כתבנו בקוד ה-VHDL.

השלב הבא בהוצאת התכנון מן הכח אל הפועל היא הסינתזה, שמפיקה תרשים חשמלי בסיסי של המעגל שייווצר בעקבות התכנון שבקוד. מדובר על תרשים לא סופי, כי לא מופיעים בו כל החיבורים החשמליים וחלקי הסיליקון שיהיו ברכיב הפיזי הסופי. לסינתזה ישנם שני חלקים:

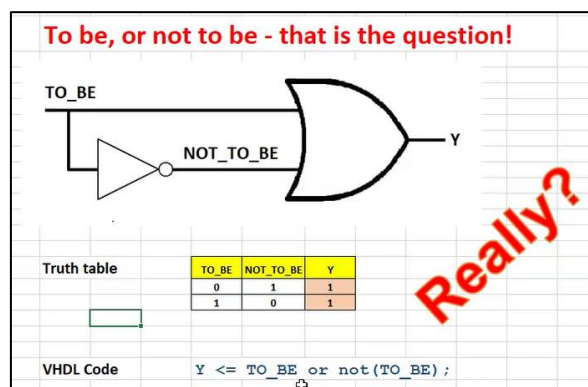
- Translation – תרגום מ-VHDL לרכיבים לוגיים שונים.
- Optimization – אופטימיזציה שמשפרת את התכנון ומתאימה אותו למטרות ואילוצים שונים.

תופעת ה-glitch בלוגיקה קומבינטורית

לוגיקה קומבינטורית היא צירוף של שערים לוגיים ושאר רכיבים, כאשר לצירוף זה ישנן כניסות ויציאות, והשפעת הכניסות על היציאות אינה מותנית בהשגחה או באירוע כלשהו, אלא מתרחשת (כמעט) בצורה מיידי. מקובל לייצג לוגיקה קומבינטורית כ'ענף' עם כניסות ויציאות:



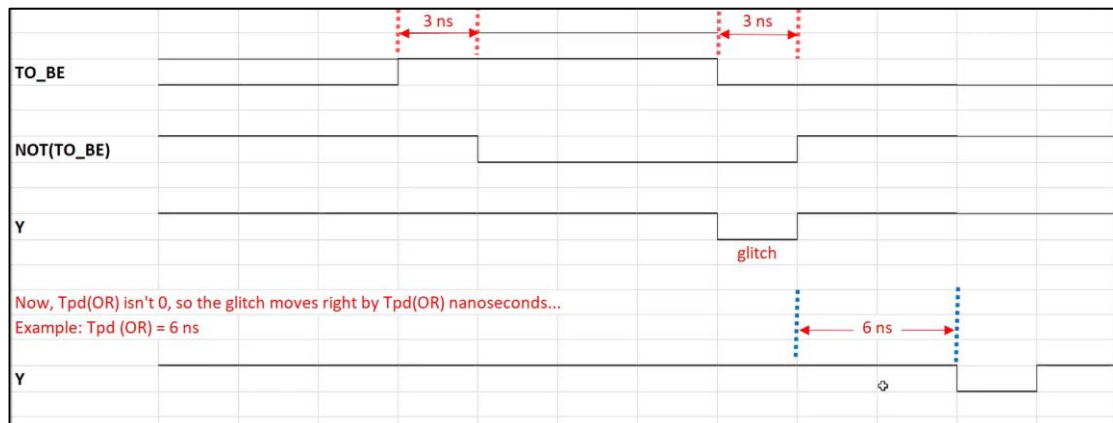
בפועל, בגלל שתהליכים פיזיקליים לוקחים זמן, יש לשערים הלוגיים ולחיווט שביניהם השהייה, והם לא יציגו ביציאה את הערך המצופה מיד עם שינוי הכניסה שלהם, אלא לאחר זמן קצר. העובדה הזו גורמת לפעמים לתופעת ה-glitch. נדגים תופעה זו באמצעות המעגל הבא, שהוא קוריוז שאינו משמש בחיים האמיתיים למטרה כלשהי:



בכניסה TO_BE נכנס ערך לוגי כלשהו, שעובר מצד אחד ישירות לתוך שער OR ומצד שני עובר דרך מהפך (שער NOT) אל תוך הכניסה השנייה של אותו שער OR. לא משנה מה יהיה הערך של הסיגנל TO_BE, תמיד ייכנסו לתוך שער OR 1 ו-0, כי אות אחד יהיה זהה ל-TO_BE והשני יהיה הופכי לו. כאשר 1 ו-0 נכנסים לתוך שער OR, הוא אמור להוציא 1. אם כן מבחינה לוגית ('בוליאנית') היציאה של שער OR שבמעגל המתואר לעיל, היא 1 באופן קבוע. אך במציאות יכול להופיע לפעמים הערך 0.

כדי להבין למה במעגל הזה יהיה ביציאה לפעמים 0, נתבונן בסט הגרפים הבא. כולם נעים על פני אותו ציר זמן, אך כל אחד מציג ערך אחר. האורך של כל משבצת מייצג 3 ננו-שניות. בתיאור הבא נתייחס רק להשהיה של השערים הלוגיים ולא להשהיה של החיווט.

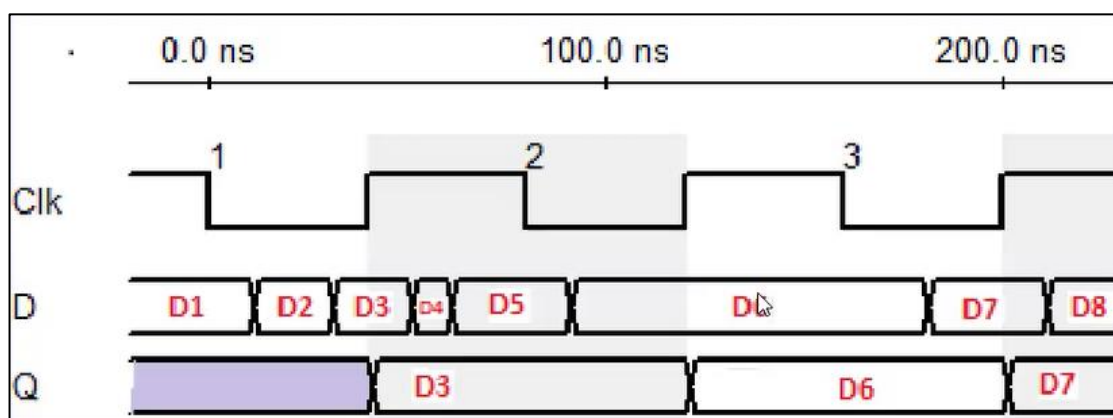
ערכי ההשהיות שמוצגות כאן הן רק דוגמה. בפועל, לכל רכיב יש את ערך ההשהיה האופייני לו, ואמור להיות מוצג בדף הנתונים של היצרן שלו.



הגרף הראשון מייצג את הערך של האות BE_TO , שמשתנה מ-0 ל-1 וחוזר אחר 12 ננו-שניות ל-0. הגרף שמתחתיו מייצג את היציאה של המהפך. מוצג בו הערך ההפוך מהערך של BE_TO , אך בהשהיה של 3 ננו-שניות (בגלל ההשהיה של שער NOT). הגרף הבא מייצג את הערך שהיה אמור להופיע ביציאת שער OR אילו תגובתו הייתה מיידית. אנו רואים שבפרק הזמן שבו בשתי הכניסות של שער OR הן 0, יציאת שער OR הייתה אמורה להציג 0. בגלל שלשער OR עצמו יש השהיה של 6 ננו-שניות, התופעה של 3 ננו-שניות שבהן הערך של היציאה שלו היא 0, לא תקרה מיד כאשר בכניסתו יש שני אפסים, אלא רק לאחר 6 ננו-שניות, כפי שרואים בגרף האחרון.

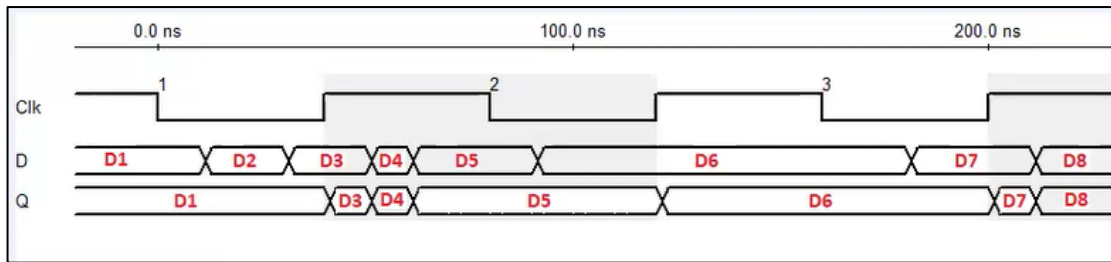
Latch מול Flip-Flop

ה-Flip-Flop הוא רכיב שמעביר מידע לוגי (0 או 1, שמיוצגים באמצעות שני רמות מתחים שונות) מהכניסה ליציאה שלו ברגע עליית שעון. המידע שיעבור ליציאה הוא בדיוק המידע שהיה בכניסה ברגע עליית השעון, ולא ישתנה עד עליית השעון הבאה. נראה דוגמה (ה'סוכריות' הן סימון מקובל בתיאור העברת מידע, ומייצגות מידע לוגי שקיים באזור כלשהי למשך פרק זמן שעל פניו נמתחת ה'סוכריה'):



היציאה Q מקבלת את הערך שהיה בכניסה D בכל עליית שעון. שינויי המידע שמתרחשים בין עליית שעון לעליית השעון הבאה לא משפיע כלל על היציאה.

לעומת ה-Flip-Flop, ישנו רכיב בשם Latch, שמעביר מידע מהכניסה ליציאה כל עוד הערך הלוגי של השעון הוא 1. בגלל שכל עוד ערך השעון הוא 1 המעבר של המידע מהכניסה ליציאה פעיל, כאשר ישתנה המידע הכניסה בזמן שהשעון הוא בעל ערך 1, היציאה תשתנה ישירות ללא צורך בעליית שעון נוספת. נראה דוגמה:



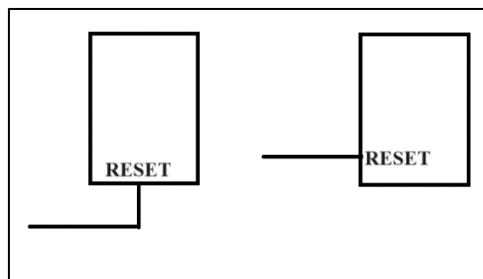
כמו שרואים, היציאה של ה-Latch משתנה בהתאם לכניסה כל עוד ערך השעון הוא 1, וכאשר ערך השעון הוא 0, היציאה 'זוכרת' את הערך האחרון שהיה בכניסה בזמן שעריך השעון היה 1.

שבוע 4

RESET

בזמן שנדלקת מערכת לוגית, אין לנו מידע על הערכים הלוגיים ברכיבי הזיכרון השונים, מה שלא מאפשר תחילת עבודה על גבי בסיס ראשוני ידוע. לשם כך, ב-Flip-Flop ישנה כניסה שנקראת RESET (ובקיצור RST), שמכניסה ליציאתו ערך שקבענו כאשר תכננו את ה-Flip-Flop (כרגיל 0). ישנם שני סוגים של RESET:

- **RESET סינכרוני** – מבצע את האתחול של ה-Flip-Flop רק בתנאי שהוא פעיל בזמן עליית שעון. בכתיבת תכנון ה-Flip-Flop מגיע בכפיפות ל-rising edge.
- **RESET א-סינכרוני** – מבצע את האתחול של ה-Flip-Flop ברגע שהוא פעיל ללא תלות בשעון. בתרשים של Flip-Flop מקובל לסמן RESET סינכרוני ככניסה הנמצאת בצדו של הריבוע המייצג את ה-Flip-Flop (כמו שמופיע בתרשים הבא מימין), ולסמן RESET א-סינכרוני ככניסה הנמצאת בתחתית הריבוע הנ"ל (כמו שמופיע בתרשים הבא משמאל).



ישנן שתי דרכים לתכנן את הפעלת ה-RESET:

- **Active high** – מבצע אתחול כאשר כניסת ה-RESET מקבלת ערך לוגי 1.
 - **Active low** – מבצע אתחול כאשר כניסת ה-RESET מקבלת ערך לוגי 0.
- בתכנון לוגי, חובה לדאוג לכך שבכל Flip-Flop יהיה RESET! אי הקפדה על כלל זה היא 'עברת תכנון'.

Clock enable

כאשר מעוניינים שלא כל עליית שעון תשפיע על רכיב שמקבל את השעון (לדוגמה: Flip-Flop), אפשר להוסיף לרכיב כניסה שתיקרא clock enable, ולהתנות בקוד ה-VHDL את כל ההשפעות של השעון בכך שכניסת clock enable פעילה (בעלת ערך שמוגדר כ'פעיל' – 1 או 0, לפי רצונו של המתכנן). עם זאת, יש להדגיש שה-RESET לא יכול להיות כפוף ל-clock enable, ויש לאפשר לו להיות פעיל בעליית שעון (אם הוא סינכרוני) גם כאשר clock enable אינו פעיל. RESET תמיד צריך בלעדיות וקדימות, כלומר לא להיות תלוי בכלום (חוץ מבשעון – אם הוא סינכרוני)!

RTL coding style

לא כל שפת VHDL ניתנת לסיומת – כלומר יכולה ליצור תכנון ספרתי. החלקים הניתנים לסיומת הם בסגנון RTL (Register Transfer Level). החלקים הללו של השפה מגדירים שני טיפוסים של תכנון: Flip-Flops ויעני' – שהוא כינוי ללוגיקה קומבינטורית, שאת משמעותה הכללית הכרנו בשבוע שעבר. החלקים הנוספים של השפה משמשים לצרכים שונים כמו למשל סימולציה (למשל הוראות שונות שאנו כותבים ב-testbench).

כתיבת קוד VHDL לתכנון Flip-Flop ללא שימוש ב-wait

כדי לתכנן Flip-Flop, שפעולתו תלויה בשעון כפי שלמדנו, אפשר להשתמש ב-wait, בדרך הבאה:

```
NICE_EXAMPLE: process
begin
    wait until rising_edge(CLK);
    Q <= D;
end process;
```

(בעקרון צריך להיות גם RESET, כפי שלמדנו, אך לא ניגע בזה כעת, כי אנו מתמקדים בנקודה אחרת). הפקודה wait until היא פקודה שניתנת לסיומת ולכן התכנון שראינו זה עתה יעבור סיומת בצורה תקינה. אך עם זאת, לפעמים נרצה מסיבות שונות לא להשתמש ב-wait. במקרים כאלה אפשר לתכנן את ה-Flip-Flop בצורה הבאה:

```
process (CLK)
begin
    if rising_edge (CLK) then
        if (RST='1') then
            Q <= '0';
        else
            Q <= D;
        end if;
    end if;
end process;
```

בגלל שאין wait יש צורך ברשימת רגישויות בתחילת ה-process, כך שה-process יתרחש בכל פעם שיש שינוי באות ה-clock.

שבוע 5

הסיומת

בשבוע 2 הכרנו את תרשימי הזרימה של הוצאת תכנון ספרתי מן הכח אל הפועל. השלב הראשון אחרי כתיבת הקוד הוא הסימולציה, שאותה עשינו באמצעות תוכנת ModelSim. במעבדות עד כה תרגלנו את תהליך הסימולציה. עתה נכיר את השלב הבא – הסיומת.

לאחר ביצוע סימולציה, שנותנת לנו את האינדיקציה לכך שהתכנון שכתבנו נותן ביציאותיו ערכים שמתאימים למה שאנו מצפים בעקבות הכנסת ערכים שונים בכניסותיו, צריך להתקדם לכיוון המימוש הפיזי של התכנון. הסיומת היא שלב ראשוני, שבו התכנון הכתוב בשפת VHDL (או שאר שפות) מתורגם למעגל חשמלי עקרוני, שמכיל שערים לוגיים ושאר רכיבים אלקטרוניים.

בשיעור ראינו הדגמה של סיומת של קוד באמצעות תוכנת Precision. מטלה 5 מיועדת להכרת הכלי והעבודה אתו שלב אחר שלב.

סוגי משתנים ב-VHDL

ישנם 7 סוגים ראשיים של משתנים ב-VHDL, המופיעים בטבלה הבאה (למשתנים שיכולים להופיע גם כמערך, מוצגת האופציה הזו בטור הימני):

Scalar type	Array type
bit	bit_vector
std_logic	std_logic_vector
boolean	
integer	
real	
time	
character	string

נדון בתכונות של כמה מסוגי המשתנים הללו.

המשתנה bit יכול להכיל '1' או '0' (הגרש הימני והשמאלי הם חלק מהותי מהכתיבה של ערך למשתנה). למשתנה זה יש חסרון, שהוא יכול להכיל רק '0' או '1', ולא את הערכים הבאים:

- ערך שמסמל 'don't care', כלומר: מצב שבו לא משנה לנו מה ערך המשתנה.
 - ערך 'High Z', שמסמל מצב שבו החוט חתוך, ואין קשר חשמלי שיכול להעביר ערך לוגי למיקום הפיזי שהמשנה הזה מייצג.
 - ערך שמסמל שעדיין לא הוצב ערך לוגי במשתנה. ערך כזה חשוב, כדי לגלות בזמן הסימולציה האם ומתי המשתנה הזה נכנס לפעילות אמיתית בתוך הרכיב.
- המשתנה std_logic, שבא כדי להשלים ערכים נצרכים נוספים מעבר ל'0' ו'1', יכול להכיל 9 ערכים:

'U',	-- Uninitialized
'X',	-- Forcing Unknown
'0',	-- Forcing 0
'1',	-- Forcing 1
'Z',	-- High Impedance
'W',	-- Weak Unknown
'L',	-- Weak 0
'H',	-- Weak 1
'-',	-- Don't care

הערך הראשון ברשימה הוא 'ללא אתחול', כלומר לא הוגדר עדיין ערך למשתנה הלוגי.

הערך 'X' אומר שלא ידוע מה הערך של המשתנה.

הערכים '1' ו'0', כשמם כן הם.

הערך 'Z' מייצג את מצב 'High Z' שהוזכר לעיל.

הערך '-' הוא 'don't care' שהוזכר לעיל.

על הערכים 'W', 'L', ו-'H' לא נרחיב כעת.

יש סוג משתנה דומה ל-std_logic, בשם std_ulogic, כאשר האות u הנוספת מייצגת את הביטוי 'unresolved'. נרחיב עליו בהמשך הסמסטר או בסמסטר הבא. יש עוד סוגי משתנים שמבוססים על std_logic.

בשפת VHDL, כללי התכנון מחייבים שימוש דווקא במשתנה מסוג std_logic ולא משתנים מסוג bit. דוגמאות לכתיבת ערכים למשתני bit ו-std_logic בודדים או מערכים שלהם:

BIT	'1' '0'
BIT_VECTOR	"0101" "11110000" x"FF00"
STD_LOGIC	'1' '0' 'X' 'H' 'U'
STD_LOGIC_VECTOR	"10Z1" "UUUUUUUU"

יש לשים לב, שהערך של משתנה בודד נכתב עם גרש משמאל וגרש מימין, והערך של מערך נכתב עם גרשיים משמאל וגרשיים מימין.

השימוש בכתובה הקסא-דצימלית (כתיבת מספר בבסיס 16) מצריך תוספת x לפני המספר שבגרשיים. כתיבה הקסא-דצימלית מיועדת דווקא למערכים שאורכם הוא כפולה שלמה של 4, לפחות בגירסאות הישנות יותר של השפה.

אם מעוניינים להכניס ערך שאינו מספר בכתובה הקסא-דצימלית, כל אות שתיכתב תוכנס לארבעת הביטים שהיא אמורה לייצג בכתובה ההקסא-דצימלית. לדוגמה: הערך "XU" x יכניס בשמונה ביטים את הערכים "XXXXUUUU".

כאשר משתמשים באותיות אנגליות כערכים ל-std_logic, יש להקפיד על אותיות גדולות (למשל: 'U' ולא 'u'), בשונה מרוב המקרים ב-VHDL, שבהם אין הבדל בין אות גדולה או קטנה. בדוגמה הבאה מופיעות כמה הצהרות על מערכי std_logic_vector והשמות של ערכים לתוכם, שכולן חוקיות ב-VHDL:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity BITE_X is
port
(
  A: out  std_logic_vector(1 to 8);
  B: out  std_logic_vector(7 downto 0);
  C: out  std_logic
);
end BITE_X;

architecture BITE_X_ARC of BITE_X is

  signal S1: std_logic_vector(7 downto 0);
  signal S2: std_logic;

begin

  A <= "10101111";
  B <= "ZZZZUUUU";
  C <= '1';
  S1 <= "00000X00";
  S2 <= S1(2);

end BITE_X_ARC;
```

כמו שרואים בדוגמה, אפשר למספר איברים של מערך מלמעלה למטה או מלמטה למעלה. כמו כן, המספר של האיבר הנמוך ביותר יכול להיות 1 או 0.

הפירוש של השורה `s2 <= s1(2);` הוא: הכנס את הערך של ביט מספר 2 של המערך s1 לתוך המשתנה s2. בדוגמה שהוצגה, מדובר על הערך X, מפני שהמספר של הביט הכי נמוך ב-s1 הוא 0, וביט מספר 2 הוא השלישי מימין.

המשתנה מסוג Boolean יכול להכיל את אחד הערכים true או false, שאינם זהים ל'0' ו-'1'.

ביטוי כמו ('1' = c) מחזיר ערך true אם הוא נכון, ו-false אם הוא שגוי.

מומלץ לא להשתמש במשתנה מסוג Boolean לצורך סינתזה.

משתנה מסוג integer ב-VHDL מכיל מספר בגודל של עד 32 ביטים, שהביט הגבוה מבינם מייצג את הסימן של המספר (שלילי או חיובי). למעשה הערכים שהוא יכול להכיל הם הערכים שבין 2 מיליארד בערך לבין מינוס 2 מיליארד בערך.

אם מגדירים משתנה בתור integer, הסינתזה תגדיר אותו בתור 32 ביט. יש דרך להגביל את מספר הביטים אם מגדירים את טווח המספרים שהוא אמור להכיל. באופן כללי, לא כדאי להשתמש ב-integer לצורך סינתזה.

כל הערכים הבאים הם חוקיים עבור integer:

```
+3
-728
+123_456
16#FF00#
```

בשורה השלישית מופיעה צורת כתיבה שבה נוסף תו _ בין כל 3 ספרות, לצורך נוחות (כמו פעמים רבות לשם פסיק בין כל 3 ספרות של מספר ארוך). שפת ה-VHDL יודעת להתעלם מהתו הזה.

בשורה הרביעית מופיעה הצורה לכתיבת מספר ל-integer בכתיבה הקסא-דצימלית.

המשתנה real יכול להכיל מספר עם נקודה עשרונית (מקביל ל-float בשפת C). המשתנה הזה לא ניתן לסינתזה.

המשתנה time מכיל ערכי זמן שונים. גם הוא לא ניתן לסינתזה.

משתנים שלא ניתנים לסינתזה מיועדים לסימולציה. אפשר להשתמש בהם בכתיבת testbench.

שבוע 6

המשך נושא סוגי משתנים ב-VHDL

דוגמה לשימוש במשתנה מסוג time (שהכרנו בסוף המפגש בשבוע שעבר):

```
library IEEE;
  use IEEE.std_logic_1164.all;

entity OSCILATOR is
  generic
  (
    CLK_PERIOD: time := 100 ns
  );
  port
  (
    CLK: out std_logic
  );
end OSCILATOR;

architecture OSCILATOR_ARC of OSCILATOR is
  signal CLK_INTERNAL: std_logic := '0';
begin
  THE_NICE_THING: process
  begin
    wait for (CLK_PERIOD / 2.0);
    CLK_INTERNAL <= not (CLK_INTERNAL);
  end process;

  THE_OUTPUT: CLK <= CLK_INTERNAL;
end OSCILATOR_ARC;
```


המשתנה CLK_PERIOD שהוגדר בשורה החמישית, מכיל את אורך מחזור השעון, ובשורה החמישית לפני הסוף משתמשים בו עבור הפקודה לשינוי ערכו הלוגי של שעון לפי קצב השעון שמתאים לזמן המחזור שנקבע. הסיבה שהוא מחולק ב-2 היא כי בכל מחזור שעון מתחלף הערך הלוגי פעמיים – מאפס לאחד וחזר לאפס.

המשתנה מסוג character מכיל תו בודד (אות, ספרה או סימן כלשהו). משתנה מסוג זה אינו ניתן לסינתזה, אך הוא יכול להיות שימושי בזמן סימולציה. מערך של משתנים מסוג זה, שמכיל סדרת תווים, נקרא string. בשורה הבאה מופיעה דוגמה להצהרה על string:

```
signal STRING_SIG: string(1 to 7);
```

המספור שמופיע בסוגריים כאשר מצהירים על string צריך להתחיל דווקא ב-1 ולהיות דווקא מ-1 ומעלה, ולא מהמספר הגבוה מטה לכיוון 1, זאת בשונה מהצהרות על logic_std, שיכולות להתחיל ב-0 או ב-1 ויכולות להיות ממעלה למטה או ממטה למעלה, כפי שלמדנו.

אם נרצה להכניס ב-string מילה קצרה יותר מאורכו, נצטרך להוסיף רווחים, כך שהמחרוזת שנכניס תהיה באורך המדויק. לדוגמה (בהתבסס על הדוגמה שלפני כמה שורות):

```
STRING_SIG <= "good  ",
               "bad   " after 100 ns,
               "somehow" after 200 ns;
```

זאת משום ששפת VHDL היא strongly typed, ובכל השמה במשתנה צריך התאמה מוחלטת המשתנה שמבצעים השמה לתוכו לבין מה שמכניסים לו. דבר זה נכון עוד יותר למקרה של השמה ממשתנה מסוג אחד לתוך משתנה מסוג אחר שאינה חוקית ב-VHDL.

סדרי קדימויות בפעולות לוגיות ומתמטיות ב-VHDL

כמו באלגברה, שבה סוגריים קודמים להכל, אחר כך חזקות, אחר כך כפל וחילוק ולבסוף חיבור וחיסור, כך גם בשפות שונות יש כללי קדימויות לפעולות השונות. בטבלה הבאה מופיע סדר הקדימויות של VHDL:

Precedence	Type	Operator	Operation	Operands
highest	miscellaneous operators (arithmetic) (logical)	**	exponentiation	integer exponent
		abs not	absolute value complement	any numeric same type
	multiplying operators	*	multiplication	same type
		/ mod rem	division modulus remainder	integer integer
	sign operators	+ -	unary plus unary minus	same type same type
	adding operators	+ - &	addition substruction concatenation	same type same type
	shift operators	sll srl sla sra rol ror	Shift left logical Shift right logical Shift left arithmetic Shift right arithmetic Rotate left logical Rotate right logical	
	relational operators	= != < <= > >=	equal not equal less than less than or equal greater than greater than or equal	same type same type same type same type same type same type
lowest	logical operators	and or nand nor xor xnor	logical and logical or complement of nand complement of or logical exclusive or complement of exclusive or	same type (nand and nor may appear only once in an expression, unless parentheses are used)

כל שורה היא קבוצת פעולות בעלות אותה דרגת קדימות. כל שורה היא בעלת קדימות מעל זו שמתחתיה. בין הפעולות שבתוך אותה קבוצה אין קדימות, ויש לסמן סוגריים כדי להבהיר לכלי הסינתזה והסימולציה את סדר הקדימויות הרצוי.

פעולות בין סוגים שונים של משתנים או ערכים

בגלל ששפת VHDL היא strongly typed, כפי שלמדנו, אי אפשר לעשות פעולות בין סוגים שונים של משתנים או ערכים. בטבלה הבאה מופיעות 3 דוגמאות, הראשונה אינה חוקית, ושתי הנוספות הן מסוג משתנה מסוים, ואי אפשר לבצע השמה שלהן לתוך משתנה מסוג אחר:

$1 + 1.0$			x
$1 + 1$		integer	#
$1.0 + 1.0$		real	#

איחוד משתנים או מערכים למערך אחת (concatenation)

האופרטור & בשפת VHDL מחבר בין משתנים בודדים או מערכים, והופך אותם למערך אחד. לדוגמה, הפעולה הבאה: '1' & "0101" תייצר את המערך "01011". יש גם פעולה הפוכה, שנקראת slice, ובה לוקחים רק חלק ממערך קיים. לדוגמה, אם יש מערך בשם YYY שאורכו 8 ביטים, יהיה אפשר לעשות איתו את הפעולות הבאות:

```
YYY(5 downto 2) <= A;
B <= YYY(6 downto 3);
```

הפעולה הראשונה מציבה את ערכי המערך A בתוך קטע מתוך המערך YYY והפעולה השנייה מציבה קטע מתוך המערך YYY בתוך המערך B.

הביטוי <=

ביטוי זה יכול להתפרש במשמעויות שונות, שאותן הכלים יבינו לפי ההקשר. לדוגמה, פירושו של הביטוי הבא: $A \leq B \leq C$; הוא: "הכנס ל-A את הערך הבוליאני של הביטוי B קטן או שווה ל-C". צמד התווים $\leq$ מופיע בדוגמה זו פעמיים באותה שורה, פעם במשמעות של השמה לתוך משתנה, ופעם במשמעות של 'קטן או שווה'. הביטוי מתפרש בכל פעם לפי ההקשר שלו בתוך השורה⁴.

שתי צורות של כתיבת קוד ב-VHDL

ישנן שתי צורות לכתיבת ב-VHDL:

- Concurrent statements – תיאור של חומרה שפועלת במקביל.
- Sequential statements – תיאור של חומרה שפועלת צעד אחר צעד.

כל process בפני עצמו מתנהל במקביל לשאר הקוד, ולכן היחס בין processes שונים, או בינם לבין הצהרות שנכתבו מחוץ ל-processes, הוא יחס של concurrent statements. מכאן נגזר, שאין משמעות לסדר הכתיבה של ה-processes השונים, זאת אף אם יש תלות ביניהם, כגון שאחד האותות שמושפע ב-process אחד מסיגנלים שונים גם משפיע ב-process אחר על סיגנלים מסוימים⁵. לעומת זאת, מה שקורה בתוך ה-process נחשב sequential statements. המשמעות של פעולה צעד אחר צעד אינה כמו בתוכנה, שבה פעולות מתבצעות לפי תזמון של שעון המערכת, אלא לפי תזמונים שנדבר עליהם בהזדמנות אחרת.

השימוש בתנאי if, else ו-elsif

השימוש בתנאי if ו-else ב-VHDL נעשה רק בתוך process. בדוגמה הבאה יש דוגמה לשלוש אפשרויות לשימוש בתנאי if, else ו-elsif (שהוא קיצור של else if, שמשמעותו הוא: כאשר התנאי הקודם לא מתקיים, אם מתקיים תנאי אחר, פעולה מסוימת תתבצע).

⁴ כמאמר התנאי "דבר הלמד מעניינו ודבר הלמד מסופו".
⁵ כאשר כלי סימולציה מדמה ריצה של תכנון, הוא עובר על ה-processes בכעין 'קרוסלה' מחזורית לפני שהוא קובע את ערכי הסיגנלים בזמני הריצה, כדי להתחשב בתלות ביניהם.

IF boolean_condition THEN sequential_statements END IF;	IF boolean_condition THEN sequential_statements ELSE sequential_statements END IF;	IF boolean_condition THEN sequential_statements ELSIF boolean_condition THEN sequential_statements ELSE sequential_statements END IF;
---	--	---

בדוגמה אפשר לראות כמה מהכללים לשימוש בתנאים :

- אחרי if או elsif מופיע התנאי ואחריו המילה THEN.
- בסוף תנאי או סידרת תנאים (כולל ה-else שלהם) תופיע השורה END IF.
- הדברים שאמורים לקרות אם... או אם לא... מופיעים בשורה שאחרי התנאי ובהזחה.

השימוש ב-case

אפשרות נוספת להתנות חלקים של הקוד בתנאים כלשהם היא באמצעות שימוש ב-case. באפשרות זו מתייחסים למשתנה או לביטוי כלשהו, ולאחר מכן כותבים מה לעשות עבור כל ערך אפשרי שלו. נבין יותר באמצעות הדוגמאות הבאות :

CASE object IS WHEN choice1 => sequential_statements; WHEN choice2 => sequential_statements; * * * END CASE;	CASE object IS WHEN choice1 => sequential_statements; WHEN choice2 => sequential_statements; * * * WHEN OTHERS => sequential_statements; END CASE;
--	---

נתחיל עם הדוגמה השמאלית :

- בתחילת השורה הראשונה מופיעה המילה השמורה CASE, לאחר מכן אובייקט כלשהו (למשל שם של משתנה) ולבסוף המילה IS.
- בשורה הבאה מתחילה רשימה של אפשרויות לערכו של הביטוי, כשכל שורה מתחילה ב-WHEN, לאחר מכן ערך מתוך הערכים האפשריים, לאחר מכן הסימן => ולבסוף התוויות מסוימות שאמורות להתקיים אם הערך של האובייקט הוא האפשרות שעליה מדובר.
- בסיום האפשרויות יש לכתוב בשורה נוספת END CASE.

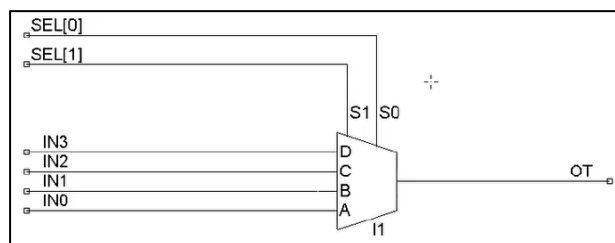
הדוגמה הימנית דומה לשמאלית, אך מודגמת בה האפשרות שלא לציין בפירוש את כל האפשרויות, אלא לכלול את כל האפשרויות שלא נכתבו בפירוש באמצעות הביטוי WHEN OTHERS, שפירושו : כאשר אף אחת מהאפשרויות שמנינו עד כה לא התקיימה. כאשר לא מפרטים את כל האפשרויות בפירוש, יש לכתוב WHEN OTHERS ולכתוב מה קורה בשאר המצבים. דוגמה למצב שבו לא מפורטות כל האפשרויות היא שימוש ב-case המתייחס למשתנה מסוג std_logic ללא התייחסות מפורשת לכל תשעת הערכים האפשריים עבור כל משתנה כזה (כמו שלמדנו בשבוע הקודם, בפירוט לגבי std_logic). בהמשך השיעור נלמד מה קורה כשלא מפרטים את כל האפשרויות.

השוואה בין שימוש ב-case לבין שימוש ב-if,else

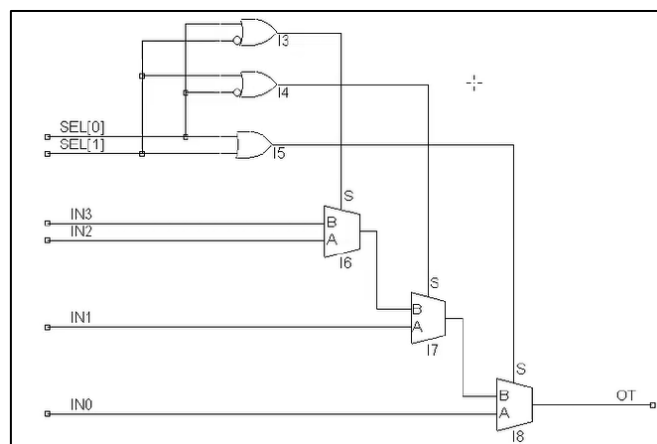
שני קטעי הקוד הבאים שווים זה לזה מבחינה לוגית. הימני משתמש ב-case והשמאלי ב-if,else,elsif :

<pre> library IEEE; use IEEE.std_logic_1164.all; entity MUX4_1 is port(SEL: in std_logic_vector(1 downto 0); IN0: in std_logic; IN1: in std_logic; IN2: in std_logic; IN3: in std_logic; OT: out std_logic); end MUX4_1; architecture IFELSE_ARC of MUX4_1 is begin P0: process(SEL, IN0, IN1, IN2, IN3) begin if (SEL = "00") then OT <= IN0; elsif (SEL = "01") then OT <= IN1; elsif (SEL = "10") then OT <= IN2; elsif (SEL = "11") then OT <= IN3; else OT <= 'X'; end if; end process; end IFELSE_ARC; </pre>	<pre> library IEEE; use IEEE.std_logic_1164.all; entity MUX4_1 is port(SEL: in std_logic_vector(1 downto 0); IN0: in std_logic; IN1: in std_logic; IN2: in std_logic; IN3: in std_logic; OT: out std_logic); end MUX4_1; architecture CASE_ARC of MUX4_1 is begin P0: process(SEL, IN0, IN1, IN2, IN3) begin case SEL is when "00" => OT <= IN0; when "01" => OT <= IN1; when "10" => OT <= IN2; when "11" => OT <= IN3; when OTHERS => OT <= 'X'; end case; end process; end CASE_ARC; </pre>
--	--

אמנם מבחינה לוגית אין הבדל ביניהם, אך בגלל ש-case הוא סוג של תפריט, ותנאי if,else הם שרשרת של שלבים זה על גב זה, יהיה הבדל בתוצאות הסינתזה של שני הקודים. הקוד הימני יוציא בסינתזה את המעגל הבא שמבוסס על MUX (רכיב שנכיר בהמשך):



הקוד השמאלי לעומתו, יוציא בסינתזה את המעגל הבא, הבנוי כדרגה על גבי דרגה עם סדר קדימויות:



יש להדגיש שגם אחרי ההבדל בסינתזה שני הרכיבים שווים זה לזה מבחינה לוגית. ההבדל הוא במבנה הפנימי של המעגל. בהמשך נבין במה ההבדל הזה יהיה משמעותי.

שימוש בתנאי if ללא else או שימוש ב-case ללא התייחסות לכל האפשרויות

מצב כזה יוציא בסינתזה latch (שכולל גם ביט או ביטים של זיכרון) ולא לוגיקה קומבינטורית, כפי שהיינו רוצים. בסוף השיעור של שבוע 3 הכרנו את שני המושגים הללו.

כתיבה מקוצרת ללא המעטפת המלאה של process

בשפת VHDL, ישנם שלושה מקרים שבהם אפשר לוותר על הכתיבה המפורשת של process, ולכתוב תיאורי חומרה מחוץ ל-process.

• מקרה של Target <= expression

כאשר סיגנל מסוים מקבל תוצאה של ביטוי לוגי או מתמטי, אין צורך לכתוב את השורה המתארת זאת דווקא בתוך process. לדוגמה, שני המבנים הבאים זהים זה לזה מבחינת היחס של VHDL אליהם:

Concurrent form	Sequential form
<pre>architecture AND2_ARC_V1 of AND2 is begin Y <= A and B; end AND2_ARC_V1;</pre>	<pre>architecture AND2_ARC_V2 of AND2 is begin process(A, B) begin Y <= A and B; end process; end AND2_ARC_V2;</pre>

• מקרה של conditional signal assignment

במקום לציין בפירוש הדרגה של if ו-else, יש אפשרות לעשות פעולה שתלויה בפרמטרים שונים בצורה מקוצרת, תוך שימוש ב-when ו-else. לדוגמה, מתואר כאן קוד שמכניס ל-Z את הערך של A – אם X גדול משלוש, את הערך של B – אם X קטן משלוש, או את הערך של C – אם שני התנאים לא מתקיימים (כאשר X שווה לשלוש):

```
Z <= A when (X > 3) else
      B when (X < 3) else
      C;
```

syntax כזה יכול להיות בשימוש רק מחוץ ל-process, ולא בתוכו.

• מקרה של selected signal assignment

במקום להשתמש ב-case, יש אפשרות לעשות פעולה שתלויה בפרמטרים שונים בצורה מקוצרת, תוך שימוש ב-when. בדוגמה הבאה מתוארת השמה שמותנית בערכו של הביטוי (הדו-ספרתי) CHOICE:

```
with CHOICE select
  Z <= A   when "00",
        B   when "01",
        C   when "10",
        D   when "11",
        'X' when OTHERS;
```

גם syntax כזה יכול להיות בשימוש רק מחוץ ל-process, ולא בתוכו. יש לשים לב שגם כאן יש להשתמש ב-OTHERS כאשר לא מצוינות כל האופציות בפירוש.

דוגמאות למבנים נפוצים של לוגיקה קומבינטורית

הדוגמה הראשונה שנראה היא DECODER, שמקבל מספר בינארי מסוים (בדוגמה זו הוא דו-ספרתי), ומוציא ערך '1' ביציאה שמספרה שווה לערך המספר הבינארי וערך '0' בשאר היציאות:

```

Library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

Entity DECODER is
port
(
  ADRS: in std_logic_vector(1 downto 0);
  Sel0, Sel1, Sel2, Sel3: out std_logic
);
end DECODER;

architecture AAA of DECODER is
begin
  Sel0 <= '1' when (ADRS = "00") else '0';
  Sel1 <= '1' when (ADRS = "01") else '0';
  Sel2 <= '1' when (ADRS = "10") else '0';
  Sel3 <= '1' when (ADRS = "11") else '0';
end AAA;

```

הדוגמה הבאה היא ENCODER, שעושה את הפעולה ההפוכה – מוציא ערך בינארי, שתואם את מספר הכניסה שבה התקבל הערך '1'.

```

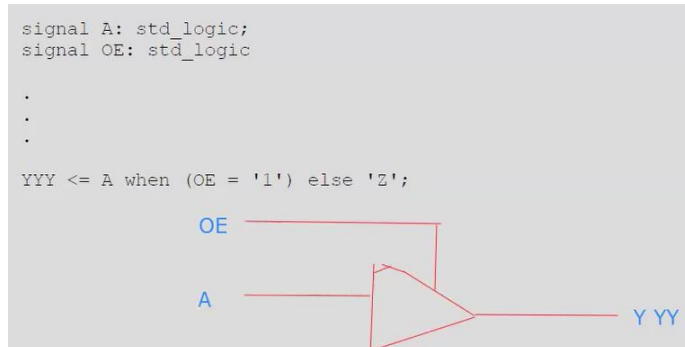
Library IEEE;
use IEEE.std_logic_1164.all;

Entity ENCODER is
port
(
  Sel0, Sel1, Sel2, Sel3: in std_logic;
  WORDOUT: out std_logic_vector(1 downto 0)
);
end ENCODER;

architecture AAA of ENCODER is
begin
  process(Sel0, Sel1, Sel2, Sel3)
  begin
    if (sel0 = '1') then
      WORDOUT <= "00";
    elsif (Sel1 = '1') then
      WORDOUT <= "01";
    elsif (Sel2 = '1') then
      WORDOUT <= "10";
    elsif (Sel3 = '1') then
      WORDOUT <= "11";
    else
      WORDOUT <= "XX";
    end if;
  end process;
end AAA;

```

בדוגמה הזו, בגלל שיש הדרגה של if ו-else, ולא שימוש ב-case, קיים סדר קדימויות. זה משפיע למשל במקרה שבו בשתי כניסות יתקבל הערך '1', שלא תהיה סתירה בין תנאים, אלא הביטוי הלוגי הראשון שיהיה אמת, יגרום לפעולה שבאה אחריו, בלי להתחשב בכך שגם ביטוי שנמצא בהמשך הוא אמת. המבנה הבא שנלמד הוא THREE-STATE (נקרא גם Tri-state או High Z). היציאה שלו (שנקראת בדוגמה YYY) מקבלת את ערך הכניסה (שנקראת בדוגמה A) אם ערך הכניסה המאפשרת (שנקראת בדוגמה OE כלומר output enable) הוא '1'. אחרת, ערך היציאה הוא 'Z' כלומר כאילו יש קטיעה של החוט. כאן מוצגת דוגמה של קוד, עם תרשים של מה שהיא מייצרת:



הדוגמה הבאה היא MULTIPLEXER ובקיצור MUX. מדובר על רכיב שמנתב את המידע מתוך אחת מכניסות המידע שלו (IN0-IN3 בדוגמה להלן) אל יציאתו. מספר הכניסה שממנה יילקח המידע שיוצג ביציאה, נבחרת באמצעות כניסה או כניסות (SEL בדוגמה להלן) שבה מוכתב מאיזו כניסה יש לנתב את המידע. הנה דוגמה לקוד שמתאר MUX:

```

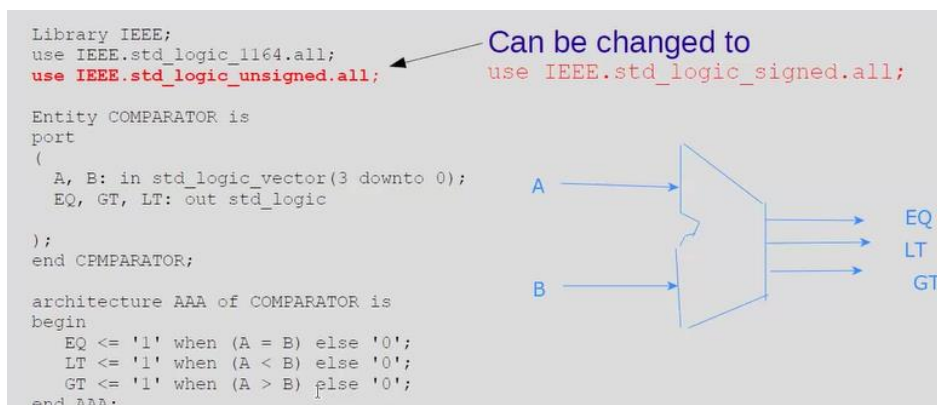
library IEEE;
use IEEE.std_logic_1164.all;

entity MUX4_1 is
port(
  SEL: in std_logic_vector(1 downto 0);
  IN0: in std_logic;
  IN1: in std_logic;
  IN2: in std_logic;
  IN3: in std_logic;
  OT: out std_logic
);
end MUX4_1;

architecture MUX4_1_ARC of MUX4_1 is
begin
  P0: process(SEL, IN0, IN1, IN2, IN3)
  begin
    case SEL is
      when "00" => OT <= IN0;
      when "01" => OT <= IN1;
      when "10" => OT <= IN2;
      when "11" => OT <= IN3;
      when OTHERS => OT <= 'X';
    end case;
  end process;
end MUX4_1_ARC;

```

הדוגמה הבאה היא COMPERATOR, שיש לו 3 יציאות: גדול, קטן ושווה. הרכיב משווה בין שני מספרים שנכנסים אליו, ומוציא ערך '1' ביציאה שמתאימה ליחס בין שני המספרים. הנה דוגמה לקוד שמתאר אותו, לצד שרטוט של הרכיב. (מקובל לציין רכיב כזה כטרנז עם חריץ משולש בבסיס הרחב):



ההערה בתחילת הקוד מתייחסת לכך שאם מעוניינים שהמערכים A ו-B ייצגו גם ערכים שליליים (בשיטת משלים ל-2 שלומדים במערכות ספרתיות), יש להשתמש בספריה עם סיומת signed במקום unsigned. הדוגמה הבאה היא מחברים או מחסרים (ADDERS & SUBTRACTERS), שמבצעים חיבור או חיסור אריתמטי (גם כאן יש אפשרות לייצוג ללא סימן או לייצוג עם אפשרות למינוס – תלוי בבחירת הספריה):


```

Library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
.
.
.
signal A, B: std_logic_vector(7 downto 0);
signal YYY: std_logic_vector(7 downto 0);
signal VVV: std_logic_vector(7 downto 0);
.
.
.
YYY <= A + B;
VVV <= A - B;

```

Can be changed to
 use IEEE.std_logic_signed.all;

דוגמה נוספת היא ALU (כלומר Arithmetic Logic Unit) שמבצעת פעולה חשבונית או לוגית על שני מערכים שנכנסים אליה. הפעולה הספציפית שתיעשה (ותוצאתה תוצג ביציאה) תלוי בערך מערך הכניסה SEL, שאחראי לבחירת הפעולה:

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
.
.
.
entity ALU is
port(
  SEL: in  std_logic_vector(1 downto 0);
  A  : in  std_logic_vector(7 downto 0);
  B  : in  std_logic_vector(7 downto 0);
  Y  : out std_logic_vector(7 downto 0)
);
end ALU;

architecture ALU_ARC of ALU is
begin
  I
  P0: process(SEL, A, B)
  begin
    case SEL is
      when "00" => Y <= A + B;
      when "01" => Y <= A - B;
      when "10" => Y <= not(A);
      when "11" => Y <= A or B;
      when OTHERS => Y <= "XXXXXXXX";
    end case;
  end process;
end ALU_ARC;

```

Can be changed by
 use IEEE.std_logic_signed.all;

הדוגמה הבאה היא מכפיל (MULTIPLIER) שמבצע הכפלה של שני מספרים:

```

Library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
.
.
.
signal A, B: std_logic_vector(7 downto 0);
signal Y : std_logic_vector(15 downto 0);
.
.
.
Y <= A * B;

```

יש לשים לב שהתוצאה תהיה ארוכה עד פי 2 מהגורמים המשתתפים, ולכן יש לדאוג ליציאה בעלת אורך כפול.

שבוע 7

דוגמאות נוספות של מבני לוגיקה קומבינטורית נפוצים

בדוגמה הבאה נלמד על מעגל לגילוי טעויות בשליחת מידע. המעגל בודק את מספר הביטים שערכם '1' מתוך המילה שהגיעה מבחוץ. אם המספר זוגי, ערך ביט היציאה Y יהיה '0', ואם המספר הוא אי-זוגי, ערכו של Y יהיה '1'. הקוד שמתאר חומרה כזו מצורף:


```

library IEEE;
use IEEE.std_logic_1164.all;

entity PARITY is
port(
    INWORD: in  std_logic_vector(3 downto 0);
    Y      : out std_logic
);
end PARITY;

architecture PPP of PARITY is
    signal P1, P2, P3: std_logic;
begin
    P1 <= INWORD(0) xor INWORD(1);
    P2 <= INWORD(2) xor P1;
    P3 <= INWORD(3) xor P2;
    Y  <= P3;
end PPP;

```

אפשר להוסיף את הביט Y למילה המקורית בתור ביט זוגיות, כדי לאפשר למי שיקבל את המילה בהמשך לגלות טעויות. אחר תוספת הביט Y, המילה החדשה חייבת להכיל מספר זוגי של ביט '1', ואם זה לא כך יודעים שיש טעות. שיטה זו יכולה לגלות טעויות רק אילו אירעה טעות אחת בלבד. אם אירעו שתי טעויות או יותר, השיטה עלולה לפספס את הטעות.

את הפעולה שעושה הקוד הזה אפשר גם לתאר בשורה אחת, כך:

```
Y <= INWORD(0) xor INWORD(1) xor INWORD(2) xor INWORD(3);
```

בדוגמה הבאה נלמד על מחבר (ADDER) מלא, עם נָשָא נכנס (carry in) ונשא יוצא (carry out). הנשא הנכנס נוצר מחיבור של ביטים נמוכים יותר והנשא היוצא הולך לתוצאה המלאה או לחיבור של סדרת הביטים הגבוהים יותר. מצורף כאן קוד לדוגמה:

```

Library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

.
.
.
signal A, B, Y : std_logic_vector(7 downto 0);
signal Cin      : std_logic;
signal Cout     : std_logic;

.
.
signal LARGER_A: std_logic_vector(8 downto 0);
signal LARGER_B: std_logic_vector(8 downto 0);
signal LARGER_Y: std_logic_vector(8 downto 0);

.
.
LARGER_A <= '0' & A;
LARGER_B <= '0' & B;

.
.
LARGER_Y <= LARGER_A + LARGER_B + Cin;
Cout      <= LARGER_Y(8);
Y         <= LARGER_Y(7 downto 0);

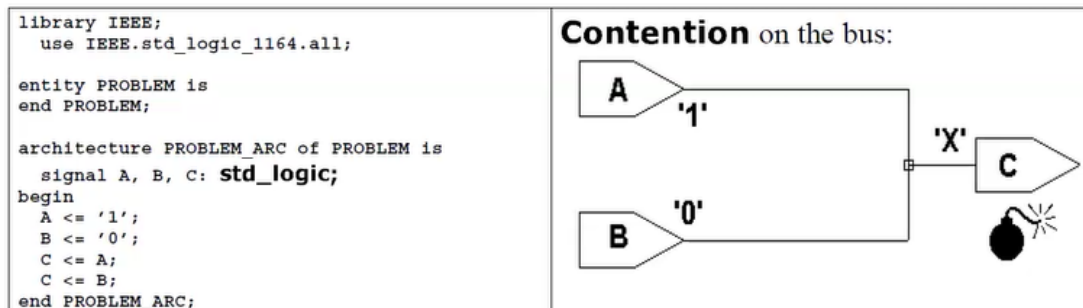
```

כל הסיגנלים בקוד הם פנימיים. בשלישיית השורות הראשונות יש הצהרות ספריות. לאחר כמה שורות שלא כתובות בפירוש מופיעות 3 שורות שמגדירות את המשתתפים בחיבור: מספרים A ו-B שאותם מחברים, מספר Y שמקבל את תוצאת החיבור, נשא נכנס שנקרא Cin ונשא יוצא שנקרא Cout. לאחר כמה שורות שלא כתובות בפירוש מופיעות 3 שורות שמגדירות משתנים מוגדלים, שעליהם יתבצע החיבור בפועל, כדי שיוכלו להכיל חיבור שכולל נשא, שחורג מעבר למספר הביטים המקורי. המשתנים נושאים את שמות המשתנים המקוריים בתוספת המילה LARGER. לאחר כמה שורות שלא כתובות בפירוש מופיעות

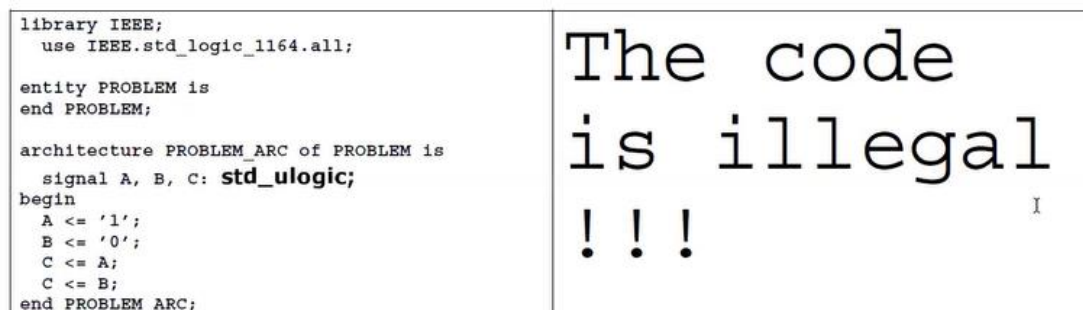
2 שורות שמתארות הכנסה של המשתנים A ו-B אל המשתנים המורחבים והוספת '0' בביט הגבוה שלהם (אי אפשר להכניס ב-VHDL מערך בגודל 8 אל תוך מערך בגודל 9, כי VHDL היא strongly typed). לאחר כמה שורות שלא כתובות בפירוש מופיעות 3 שורות: בראשונה מתבצע החיבור המלא שכולל את הנשא הנכנס Cin והתוצאה (כוללת הנשא – אם יש) נכנסת לתוך LARGER_Y, בשנייה מועבר הנשא שנוצר ב-LARGER_Y תוך כדי פעולת החיבור אל הביט Cout ובשלישית מועברת התוצאה ללא הנשא אל Y. החיבור שתואר כאן מתאים למשתני unsigned. בשיעור עמדנו על כל שב-signed יהיה צורך לבצע התאמות מורכבות יותר, כי לא כל overflow הוא באמת נשא אמיתי מבחינה מתמטית.

היחס לסתירה בין שני אותות כאשר משתמשים ב-std_logic או ב-std_ulogic

לפעמים, בעקבות תכנות לקוי, או סיבות אחרות, יכול להיות מצב שבו שני אותות שונים מנסים להיכנס לתוך אותו המיקום. מצב זה נקרא multiple drive, bus fight או contention. כאן מתוארת אפשרות שבו מצב כזה ייווצר:



יש לשים לב שהקוד הוא חוקי לפי כללי VHDL, אבל הוא ייצור חומרה גרועה. הסתירה בין שני האותות שמנסים להיכנס נפתרת באמצעות הערך 'X'. כל זה נכון כי השתמשנו כאן במשתנים מסוג std_logic. אילו היינו משתמשים במשתנים מסוג std_ulogic, התכנון לא היה חוקי, כמו שמופיע בדוגמה הבאה:



כדי להבין את הסיבה להבדל, צריך להבין שהאות u ב-ulongic פירושה 'unresolved', שהמשמעות שלה היא שסתירה כזו בכניסה היא חסרת פתרון ולא יוצב במצב כזה הערך 'X'. עבור משתני std_logic מוגדרת resolution function שמיועדת בדיוק למטרה כזו של סתירה בין ערכים. הטבלה שמגדירה את הפתרון לכל סתירה כזו מופיעה כאן:

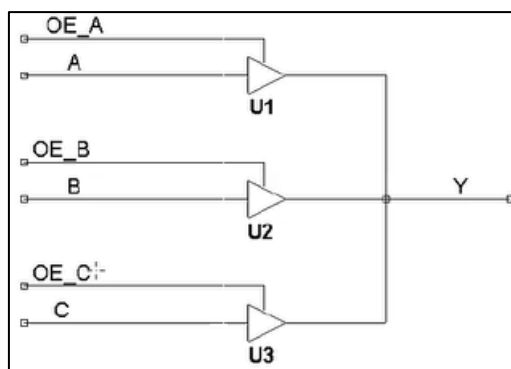
```

TYPE stdlogic_table IS ARRAY(std_ulogic, std_ulogic) OF std_ulogic;

-----
-- resolution function
-----
CONSTANT resolution_table : stdlogic_table := (
--
--      | U   X   0   1   Z   W   L   H   -   |
--
--      |-----|
--      | ('U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U'), -- | U |
--      | ('U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X'), -- | X |
--      | ('U', 'X', '0', 'X', '0', '0', '0', '0', '0', 'X'), -- | 0 |
--      | ('U', 'X', 'X', 'X', '1', '1', '1', '1', '1', 'X'), -- | 1 |
--      | ('U', 'X', '0', '0', '1', 'Z', 'W', 'L', 'H', 'X'), -- | Z |
--      | ('U', 'X', '0', '0', '1', 'W', 'W', 'W', 'W', 'X'), -- | W |
--      | ('U', 'X', '0', '0', '1', 'L', 'W', 'L', 'W', 'X'), -- | L |
--      | ('U', 'X', '0', '0', '1', 'H', 'W', 'W', 'H', 'X'), -- | H |
--      | ('U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X'), -- | - |
--      |
--      |);

```

דוגמה למצב שבו אנו נזקקים לפונקציה זו (שמתוארת בטבלה) הוא המעגל הבא, המורכב משלושה רכיבי tri-state שהכרנו בשבוע שעבר, כאשר היציאות של כולם מגיעות ל-Y:



בכל פעם, רק אחד משלושת הרכיבים פעיל, כלומר רק אחת מהכניסות OE_A, OE_B, OE_C, תהיה בעלת ערך '1'. לכן, רק אחד משלושת הרכיבים יוציא ערך לוגי שתואם את כניסתו, והשאר יוציאו 'Z'. לפי הטבלה שראינו לעיל, התנגשות של 'Z' עם '1' תיתן '1', והתנגשות של 'Z' עם '0' תיתן '0'. לכן הרכיב הפעיל הוא זה שישפיע בסוף על Y.

בגלל היתרון הזה של std_logic, ההוראה המקובלת היא להשתמש במשתנים מסוג זה בכתיבת קוד VHDL.

פין שיכול לשמש ככניסה או כיציאה

יש אפשרות להגדיר פין כך שיוכל לשמש גם כיציאה וגם ככניסה. בתרשים הבא מופיע הקוד מול שרטוט החומרה שיווצר מהקוד:

VHDL code	Hardware
<pre> entity EXAMPLE is port(D: inout std_logic; ); end EXAMPLE; architecture ARCH of EXAMPLE is signal D_IN: std_logic; signal D_OUT: std_logic; OE: std_logic; begin THE_D_IN: D_IN <= D; THE_D_OUT: D <= D_OUT when (OE = '1') else 'Z'; end ARCH; </pre>	Legend: OBUF: Tri-state output buffer IBUF: Input buffer

- בתוך הקוד, הפין מוגדר כ-inout – שילוב של in ו-out.
 - ביציאה של החומרה הפיזית יש Tri-State, כדי שכאשר הפין מוגדר בתור כניסה, לא ייכנס ל-D_IN הערך שנמצא ב-OUT_D.
 - יש לדאוג שהרכיבים החיצוניים, בין אם הם וירטואליים (כגון testbench) ובין אם הם פיזיים (כגון board שעליו מותקן רכיב FPGA שקיבל את התכנון), יוכלו לדעת מה מצב הפין ולהתנהג אליו בהתאם.
- [בהמשך השיעור ניתנו הנחיות והדרכות לביצוע מטלה 7.]

שבוע 8

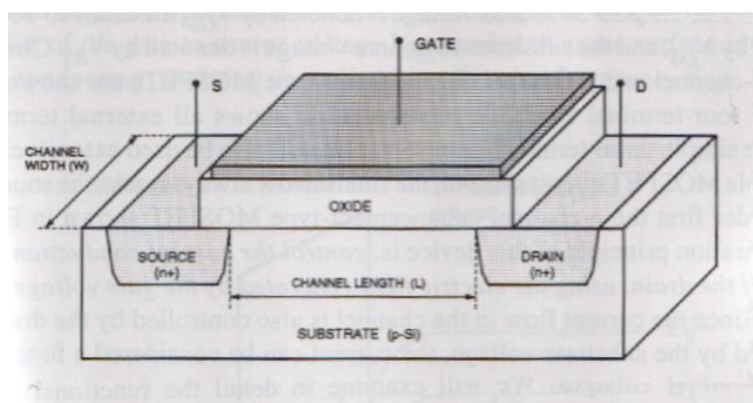
שלשה סוגי testbench

נלמד על שלשה סוגי testbench באמצעות דוגמה – בדיקה של תכנון מחבר (ADDER).

- אפשרות 1 – להזין כמה ערכים שונים בפרקי זמן שונים בכניסות המחבר, ולדגום את היציאה כדי לוודא שאכן מופיע בה חיבור אלגברי של שני המספרים שבכניסה בכל פעם.
 - אפשרות 2 (פחות מקיפה) – להזין ערך קבוע לאחת הכניסות, ולשנות רק את הערך שבכניסה השנייה למשך פרקי זמן שונים.
 - אפשרות 3 (הכי מקיפה – נקראת full factorial) – להזין את כל הצירופים האפשריים בכניסות, כל צירוף לפרק זמן מסוים, ולבדוק שתמיד יש ביציאה את התשובה הנכונה. אפשרות זו נועדה כדי לבדוק בצורה מלאה ויסודית את כל המקרים האפשריים שהמחבר יעמוד בהם בזמן אמת. היא כמובן תארך יותר זמן מהאפשרויות האחרות.
- אפשר לראות במודל קבצי VHDL של המחבר ושל שלושת אפשרויות ה-testbench, בקבצים הקשורים לפגישה 8. נרחיב יותר על הנושא הזה בסמסטר הבא.

שערים לוגיים באמצעות טכנולוגיית CMOS

כדי להבין את הטכנולוגיה, נכיר את צורת הפעולה של טרנזיסטורי MOSFET (ראשי תיבות של המילים Metal Oxide Semiconductor Field Effect Transistor).
בתרשים הבט מופיע טרנזיסטור MOSFET עם פירוט חלקיו:



הטרנזיסטור מורכב משלוש שכבות:

- מתכת (metal), שהיא כמובן מוליכה. השכבה מחוברת לפין GATE.
- תחמוצת (oxide) – למעשה מדובר על זכוכית, שאיננה מוליכה.
- מוליך למחצה (semiconductor) – כרגיל מדובר על סיליקון, שהוא מוליך למחצה. בדוגמה שמופיעה כאן, בשני הצדדים יש אזורים, שנקראים SOURCE ו-DRAIN שימזוהמים

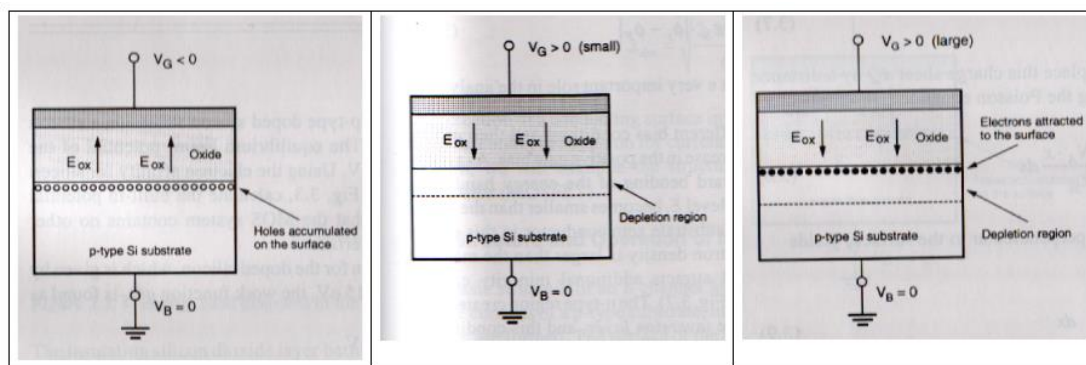
(מסוממים \ doped) בזיהום מסוג n-type (באטומים בעלי יותר אלקטרונים בקליפה החיצונית מאשר בסיליקון). מוליכי המטען באזורים אלו הם אלקטרונים. ביניהם יש אזור גדול ש'מזוהם' בזיהום מסוג p-type, שבו יש חוסר אלקטרונים, או יתר של 'חורים'.

סוג זה נקרא n-channel, כי כדי להוליד, יש צורך ליצור אזור בין ה-DRAIN ל-SOURCE שבו יוכלו לזרום אלקטרונים, שמטענם שלילי.

בגלל הזכוכית, אין הולכה ישירה בין המתכת (GATE) לסיליקון (SUBSTRATE). אבל שינוי מתחים במתכת, יחסית למתח של הסיליקון, יוצר שדה חשמלי, שמשפיע על ההולכה מימין לשמאל בתוך הסיליקון, כפי שנבין.

כאשר לא מופעל מתח ב-GATE, האזור שבין ה-DRAIN ל-SOURCE לא מוליד, כי אין בו אלקטרונים. אך אם יופעל על ה-GATE מתח חיובי, שגדול באופן ניכר ממתח הסיליקון, ייווצר בין ה-GATE לסיליקון שדה חשמלי ואלקטרונים יימשכו למעלה לכיוון ה-GATE. האלקטרונים הללו יאפשרו הולכה בין ה-SOURCE ל-DRAIN.

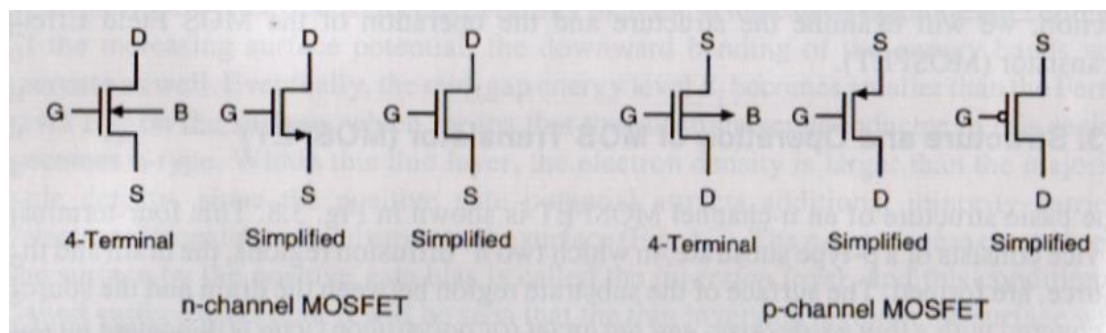
בתרשים הבא מתואר הטרנזיסטור הנ"ל בשלושה מצבים שונים (רק בימני הטרנזיסטור מוליד בין ה-SOURCE ל-DRAIN):



בדיוק כפי שפועל הטרנזיסטור שראינו כעת, כך – רק להפך, פועל טרנזיסטור MOSFET מסוג p-channel. ה-SOURCE וה-DRAIN הם בעלי סימום מסוג p, וכדי שתתאפשר הולכה ביניהם, צריך להפעיל על ה-GATE מתח שלילי, שנמוך יותר ממתח הסיליקון, ומושך אליו את ה'חורים', שיהיו מוליכי המטען בין ה-SOURCE ל-DRAIN. לסיכום:

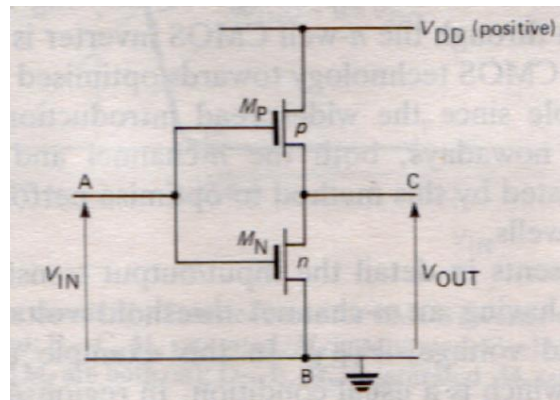
- טרנזיסטור MOSFET מסוג n-channel מוליד מה-DRAIN ל-SOURCE רק כאשר מופעל ב-GATE מתח חיובי.
- טרנזיסטור MOSFET מסוג p-channel מוליד מה-DRAIN ל-SOURCE רק כאשר מופעל ב-GATE מתח שלילי.

ישנם כמה סימונים מקובלים לשני הסוגים הללו, כפי שמופיע כאן:



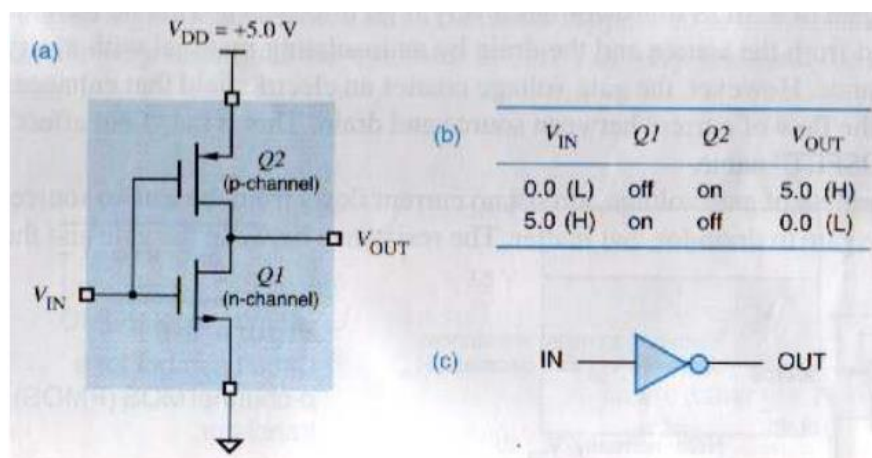
כדי לדאוג לשדה חשמלי בכיוון הנכון, גוף ה-SUBSTRATE של טרנזיסטורי n-channel מחוברים למתח ה-GROUND, כך שאילוץ מתח חיובי ב-GATE ייצור שדה בכיוון הנכון. כך גם בטרנזיסטורי p-channel, גוף ה-SUBSTRATE מחובר למתח ה-VDD (המתח החיובי של הספק), כדי שאילוץ של מתח 0 בכניסה ייצור בכיוון הנכון.

הטרנזיסטורים מסוג n-channel נקראים NMOS, הטרנזיסטורים מסוג p-channel נקראים PMOS וטכנולוגיה שבה נמצאים שני הסוגים יחד זה לצד זה נקראת CMOS – כאשר האות C משמעותה combination (שילוב בין סוגים שונים של טרנזיסטורי MOS). להרחבה של הנושא אפשר לעיין במצגת על CMOS, שנמצאת במודל בתוך הקבצים הקשורים למפגש 8. כדי להבין איך שילוב של שני סוגי הטרנזיסטורים מסוגלים ליצור שערים לוגיים, נביט בתרשים הבא, שמתאר מעגל של מהפך (שער NOT):

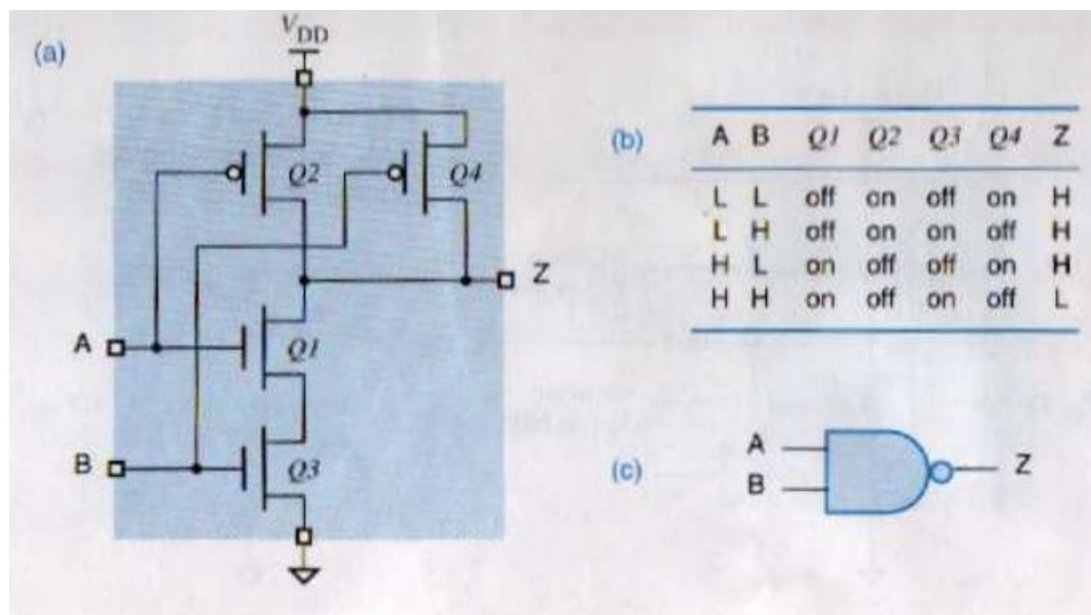


A היא הכניסה ו-C היא היציאה. המתח בכניסה מגיע לשני ה-GATES של שני הטרנזיסטורים והיציאה מחוברת למגע שבין הטרנזיסטורים.

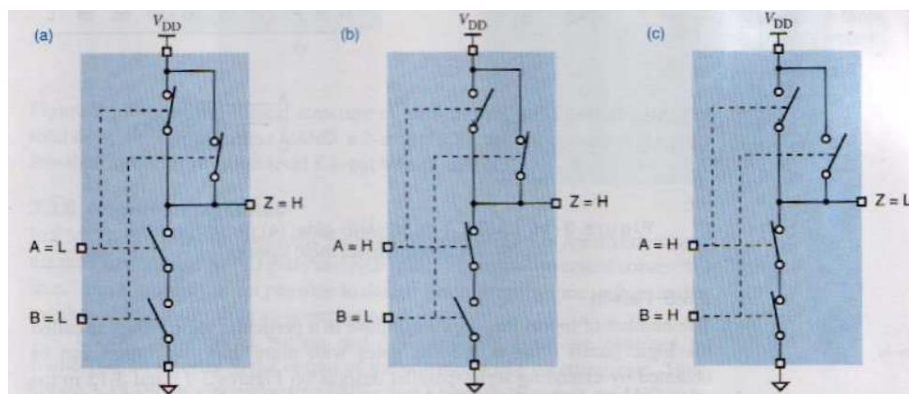
- כאשר המתח בכניסה גבוה (כמה וולטים – פעמים רבות 5 או 3.3 וולט), הטרנזיסטור העליון חסום והתחתון מוליך. אם כך, ערך מתח היציאה ישתווה (או כמעט ישתווה) ל-GROUND. כלומר, כשבכניסה יש מתח גבוה – שמייצג 1 לוגי, ביציאה יש מתח של כ-0 וולט – שמייצג 0 לוגי.
 - כאשר המתח בכניסה נמוך (כ-0 וולט), הטרנזיסטור העליון מוליך והתחתון חסום. אם כך, ערך מתח היציאה ישתווה (או כמעט ישתווה) למתח ה-VDD שהוא המתח הגבוה של המעגל. כלומר, כשבכניסה יש מתח נמוך – שמייצג 0 לוגי, ביציאה יש מתח גבוה – שמייצג 1 לוגי.
- כך נוצר מהפך – שביציאתו תמיד יהיה ערך לוגי הפוך מזה שבכניסתו. בתרשים הבא מופיע מבנה המהפך (a), הסימון המקובל שלו (c) וטבלת המיתוג' שלו (b):



טבלת מיתוג מזכירה מאד טבלת אמת, אך היא מדברת על מצבים פיזיקליים ולא על ערכים לוגיים⁶.
 בתרשים הבא מופיע המעגל של שער NAND (כלומר : not-and) – תרשים, סימון מקובל וטבלת מיתוג:

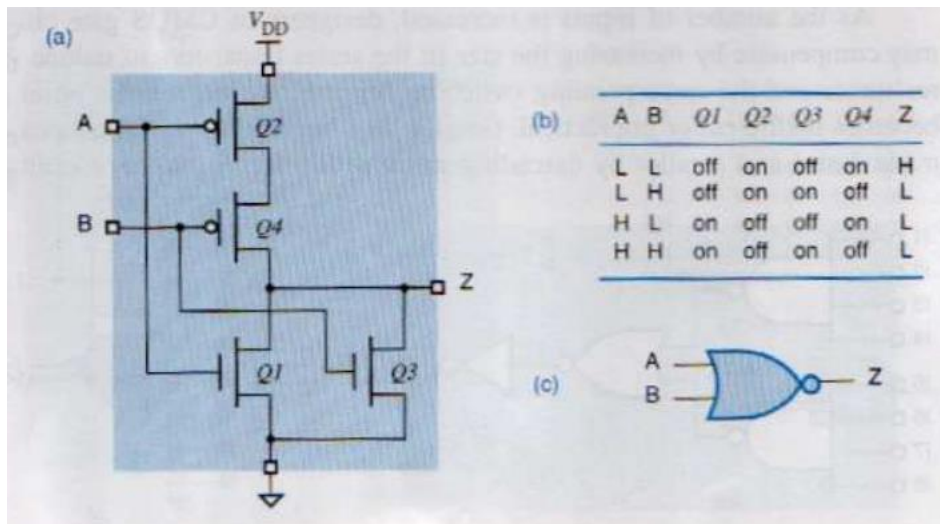


שני הטרנזיסטורים העליונים הם מסוג p-channel ושני התחתונים מסוג n-channel. כדי לגרום למתח נמוך ביציאה, חייבות שתי הכניסות להיות בעלות מתח גבוה (שמייצג 1 לוגי). רק כך ייפתחו שני הטרנזיסטורים התחתונים ויחברו בין היציאה ל-GROUND. כל רצף אחר של ערכים יחסום לפחות אחד מהטרנזיסטורים התחתונים, ויפתח לפחות אחד מהעליונים. ברגע שאחד מהעליונים פתוח, היציאה תחובר ל- V_{DD} , גם אם השני יהיה חסום. המצבים האפשריים מוצגים בתרשים הבא, שנותן הבנה על מצב המיתוג עבור כל צמד ערכי מתחים:

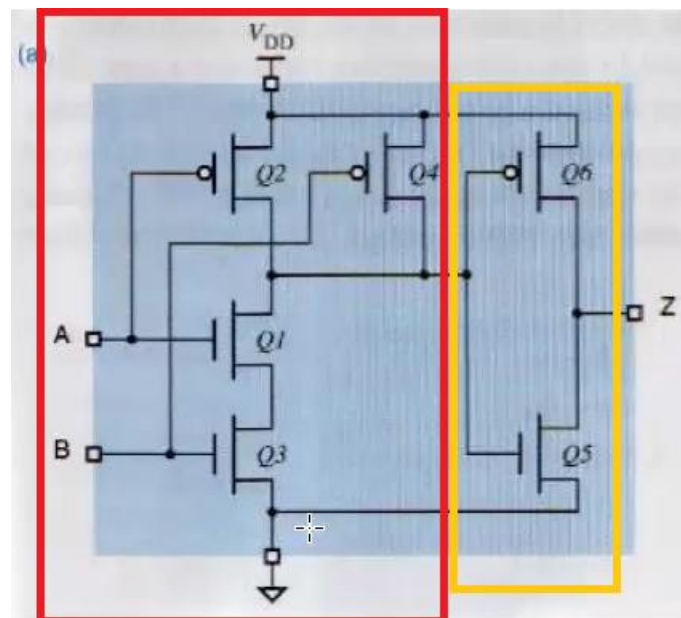


בדרך דומה – רק הפוך, פועל שער NOR, כלומר not or:

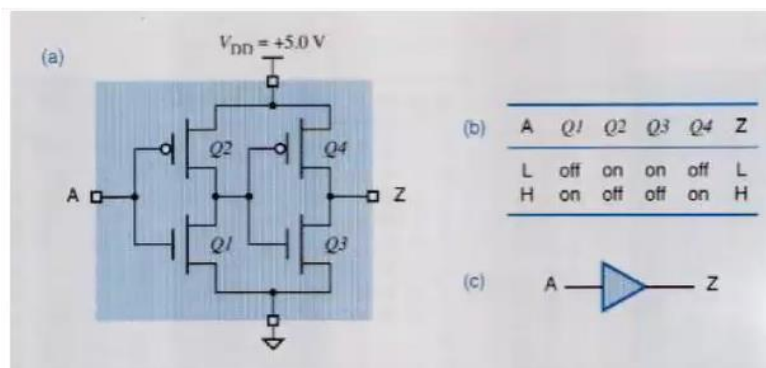
⁶ כדאי לזכור את טבלאות המיתוג. הן עלולות להיות רלוונטיות גם למבחן.



שער AND שממומש באמצעות טרנזיסטורים, מורכב משער NAND ולאחריו מהפך, כפי שמופיע בתרשים הבא:



בדומה לזה, שער OR ממומש באמצעות שער NOR ולאחריו מהפך. בתרשים הבא מתואר שער לא מהפך (non-inverting buffer):

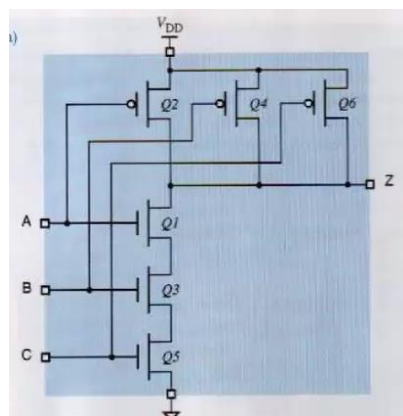


הוא בנוי למעשה משתי דרגות היפוך שבסוף מבטלות זו את זו. אולי הוא נשמע מיותר, אך משיקולים חשמליים שונים לפעמים צריך אותו.

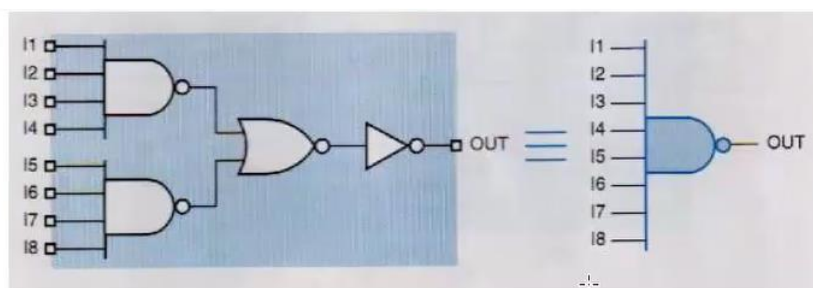
שיקולים חשמליים בבחירת שערים לוגיים למעגל

כל שער לוגי יכול להיות ממומש באמצעות שער NAND אחד או כמה. גם באמצעות שער NOR אחד או כמה, אפשר לממש כל שער לוגי.

בפועל, נעדיף שימוש בשערי NAND. נסביר את הסיבה לכך: בשני המבנים ישנו צמד טרנזיסטורים שמחובר בטור. חיבור זה עלול להוות בעיה, כי ההתנגדות של שני הטרנזיסטורים מתחברת יחד, בדומה לשני נגדים בטור. בשערי NAND, צמד הטרנזיסטורים המחוברים בטור הם מסוג n-channel, שבהם המוליכים הם אלקטרונים, שהינם בעלי מוליכות טובה יותר מאשר החורים. במילים אחרות: יש להם התנגדות קטנה יותר, וגם אחרי חיבור שלהם בטור הם יפריעו פחות לפעילות המעגל. שאלה נוספת שנגזרת מהרצון לצמצם התנגדויות גדולות בטור, היא: מה מקסימום הטרנזיסטורים שנשתמש בהם יחד באותו שער. לדוגמה, בתרשים הבא מופיע שער NAND בעל 3 כניסות:



יש אפשרות ליצור גם שערים עם יותר כניסות. עבור כל כניסה יהיה צורך להוסיף טרנזיסטור p-channel במקביל לטרנזיסטורים עליונים, וטרנזיסטור n-channel בטור עם הטרנזיסטורים התחתונים. השאלה היא מאיזה מספר של טרנזיסטורים תיווצר התנגדות שתפריע לפעולת המעגל? לצורך כך הוגדר מדד שנקרא 'fan-in' שמשמעותו: כמות הטרנזיסטורים בטור שיש בשער מסוים. כל יצרן סיליקון מגדיר מה ה'fan in' המקסימלי שיכול להיות בשער לוגי שממומש בסיליקון הספציפי, בלי לפגוע בתפקוד המעגל. כאשר יש צורך להשתמש בשערים לוגיים עם כניסות רבות, אפשר לצמצם את ה'fan in' שלהם, באמצעות מימושם כחיבור של כמה שערים. לדוגמה, בתרשים הבא מופיע שער NAND עם 8 כניסות, אבל עם 'fan in' בגודל 4:

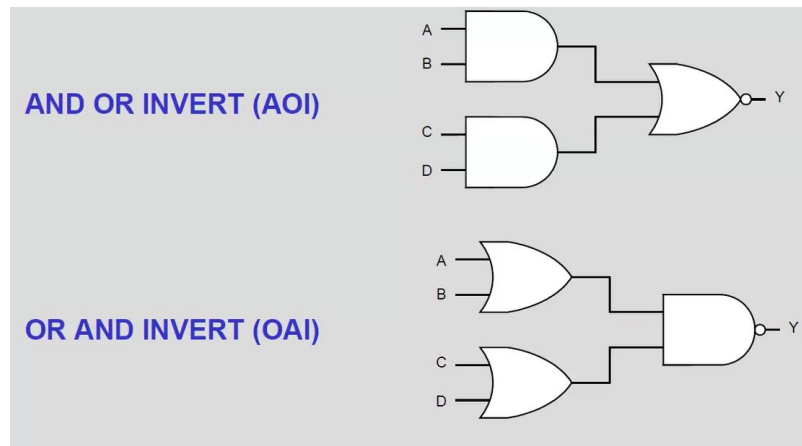


שבוע 9

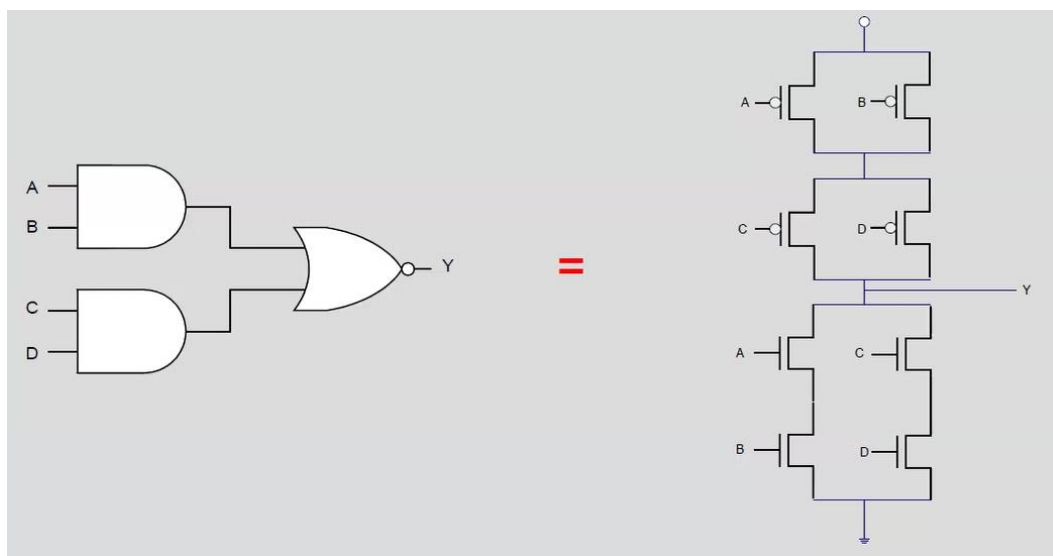
שערים לוגיים AOI ו-OAI

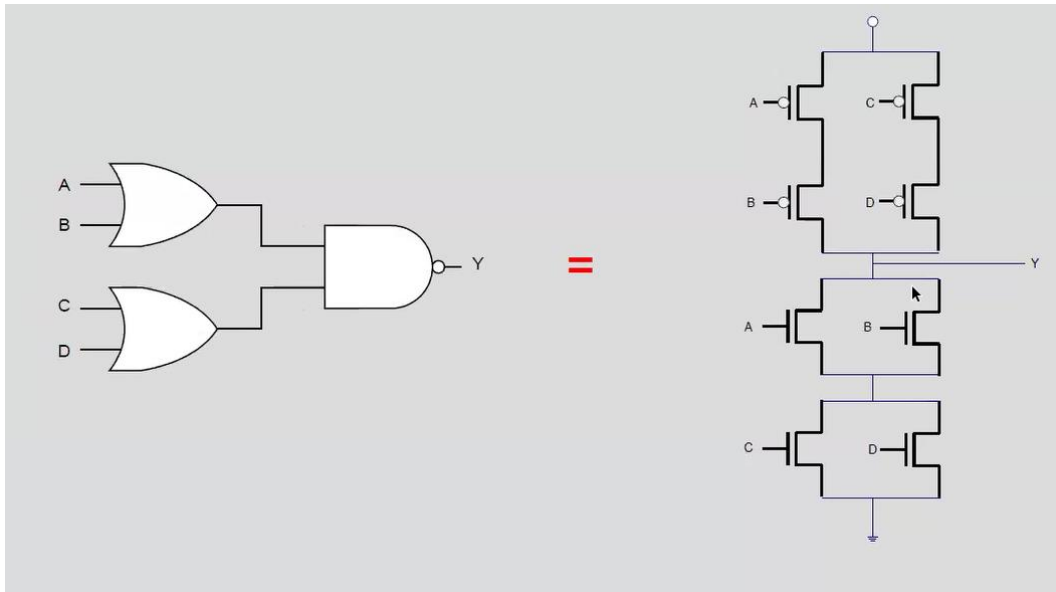
בקורס מערכות ספרתיות, למדנו שכל צירוף לוגי אפשר לייצג באמצעות סכום מכפלות (SOP – sum of products) או באמצעות מכפלת סכומים (POS – product of sums). כדי לממש עקרון זה באמצעות רכיבים פיזיים, אפשר לקחת כמה שערי AND ולהכניס אותם לשער OR (כדי להשיג SOP) או לקחת כמה שערי OR ולהכניס אותם לשער AND (כדי להשיג POS).

בגלל ששערי OR ו-AND מסובכים יותר משערי NOR ו-NAND, כפי שראינו בשבוע שעבר, נשתמש במבנים של שערי OR שנכנסים לשער NAND או בשערי AND שנכנסים לשער NOR (ונתאים את הנוסחה הבוליאנית כך שתתקבל התוצאה הרצויה).
בתרשים הבא מוצגות שתי האפשרויות הללו:

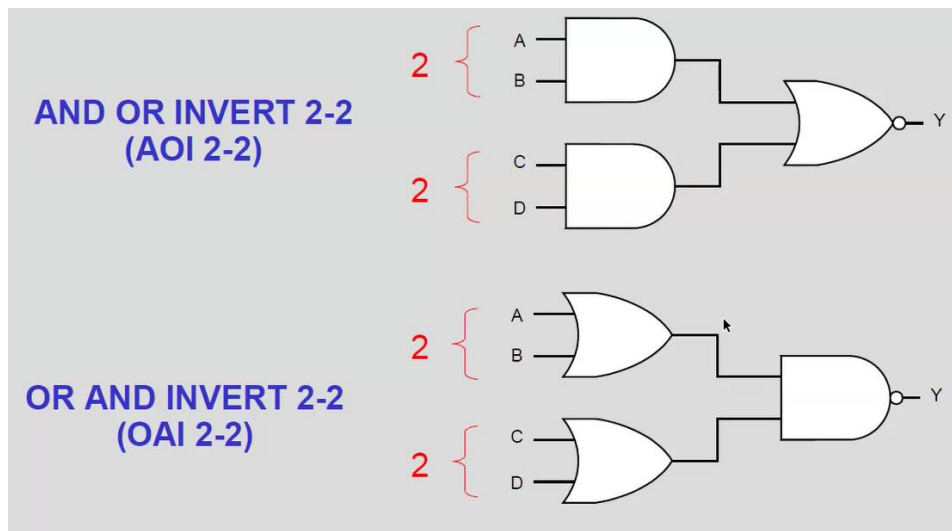


אם היינו רוצים לממש את המבנים הללו באמצעות חיבור המבנים שלמדנו בשבוע שעבר, היינו צריכים 16 טרנזיסטורים לכל מבנה: 6 לכל אחד משני השערים השמאליים ו-4 לשער הימני. אך במקום זה משתמשים במבנה אחד שעושה את הפונקציה הלוגית של שלישית השערים, אך מכיל רק 8 טרנזיסטורים. המימושים הללו מוצגים בתרשימים הבאים:

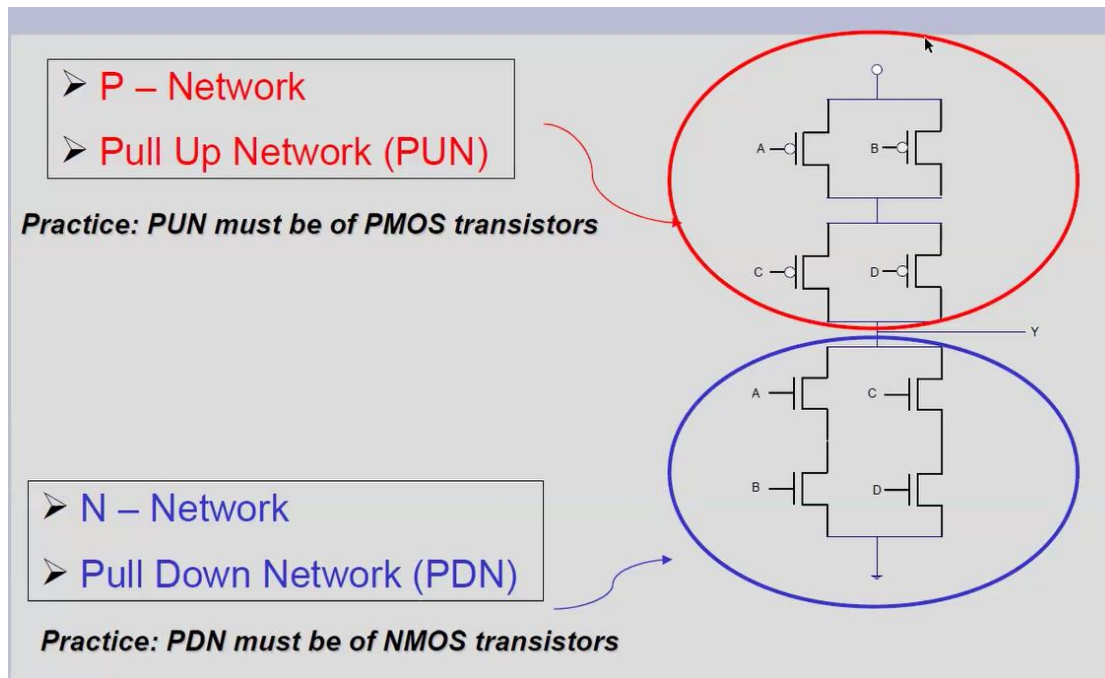




השם המלא של השערים הללו אמור לכלול מספרים שמציינים את מספר הכניסות שבשערים השמאליים.
לדוגמה:

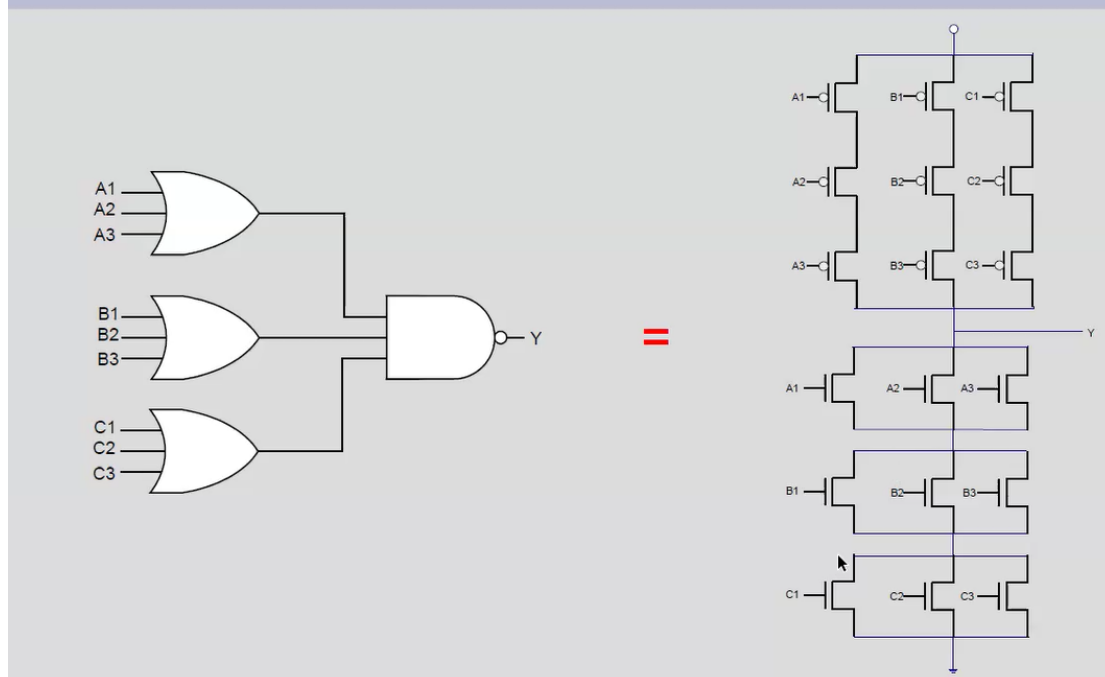


בתרשים הבא נכיר כמה מושגים ועובדות לגבי החלק העליון והחלק התחתון של שערים מסוג זה:

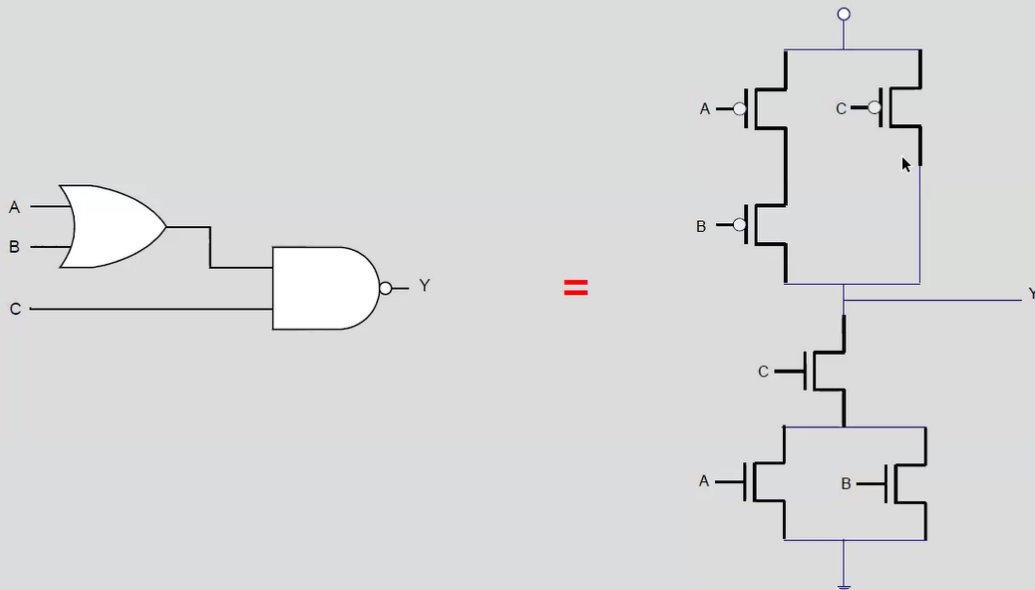


לחלק העליון מקובל לקרוא P-Network, או לחילופין Pull Up Network והוא ממומש באמצעות טרנזיסטורי p-channel. לחלק התחתון מקובל לקרוא N-Network, או לחילופין Pull Down Network והוא ממומש באמצעות טרנזיסטורי n-channel. בתרשימים הבאים מתוארות שתי דוגמאות נוספות של שערים מורכבים:

OR AND INVERT (OAI) 3-3-3

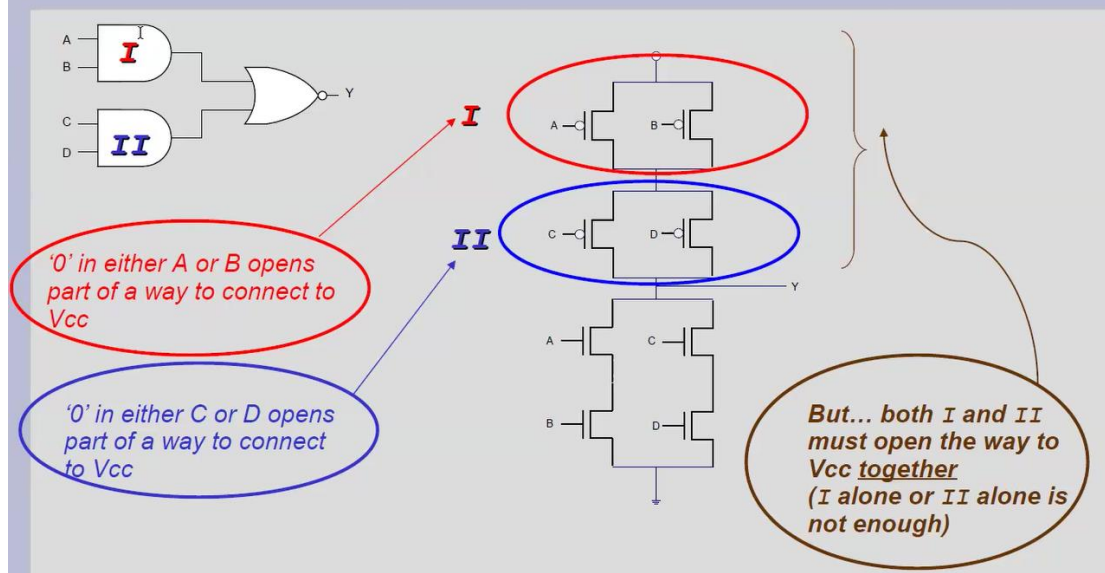


OR AND INVERT (OAI) 2-1

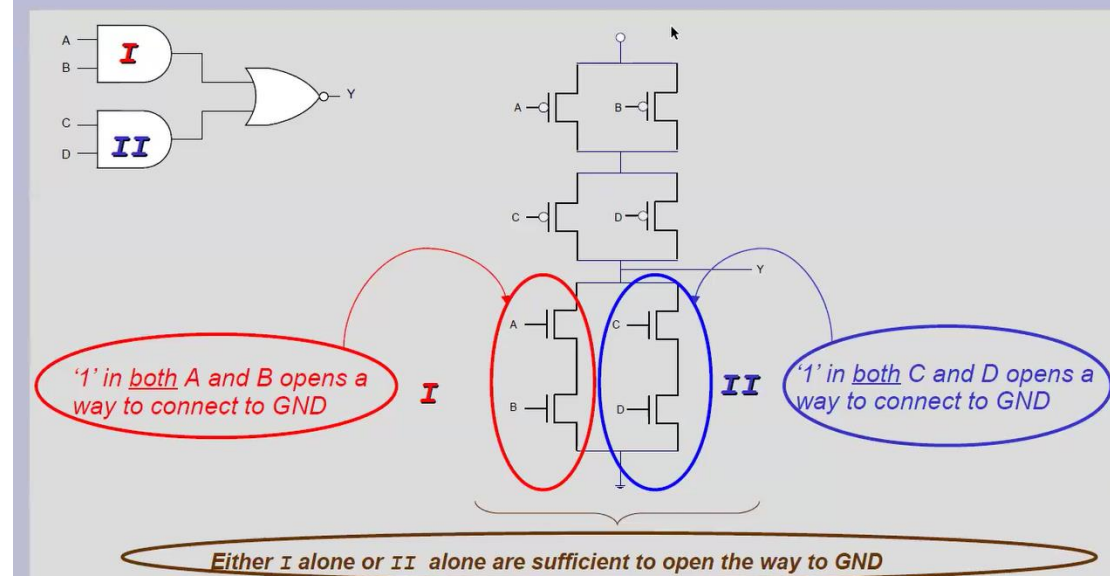


בארבעת שקפים הבאים מוסברת דרך הפעולה של מבנה AOI ושל מבנה OAI :

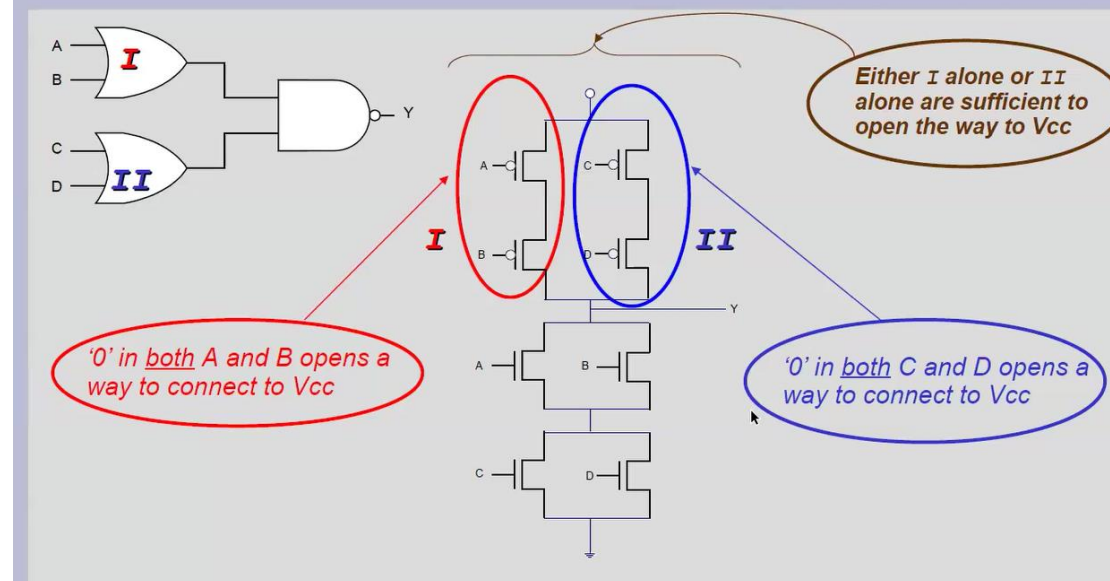
Principles: Transistors Organization in AOI 2-2 P-Network



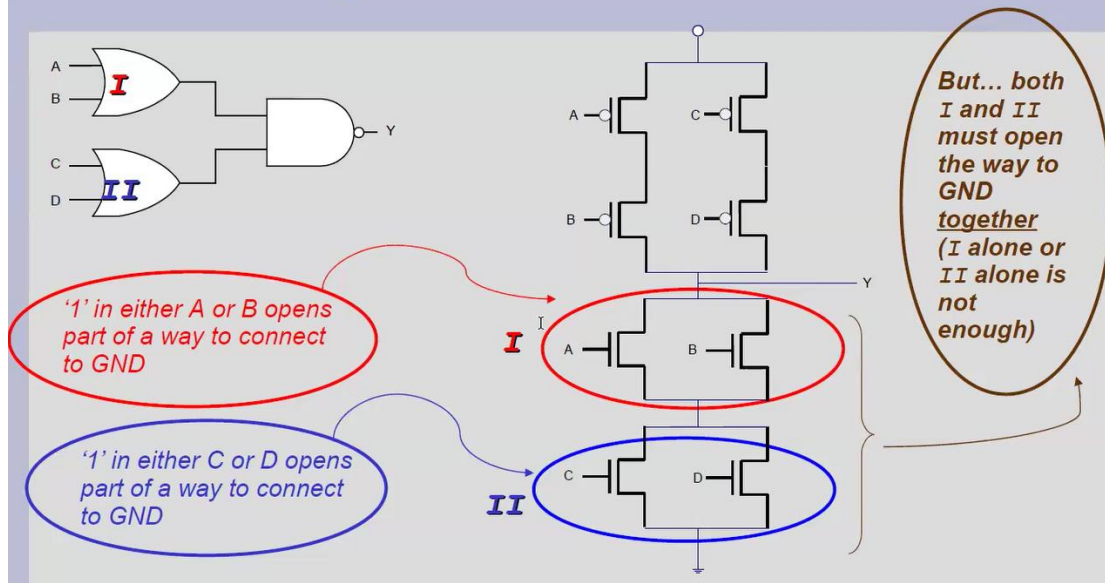
Principles: Transistors Organization in AOI 2-2 N-Network



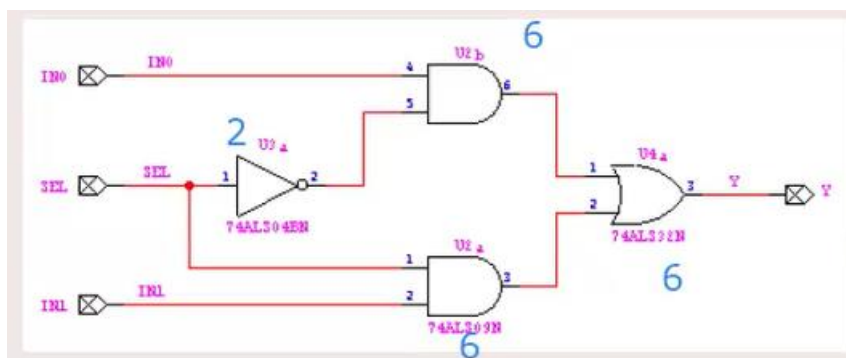
Principles: Transistors Organization in OAI 2-2 P-network



Principles: Transistors Organization in OAI 2-2 N-Network



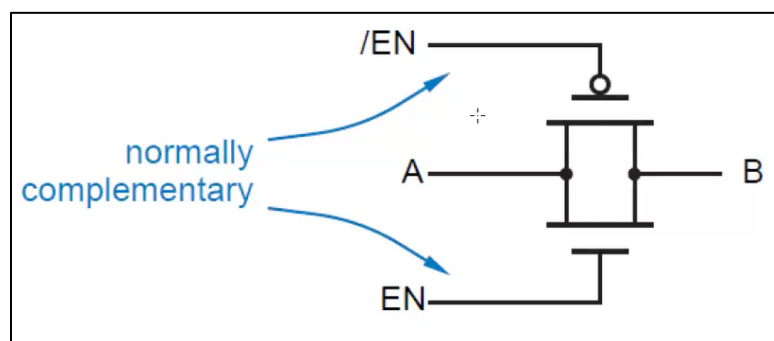
נראה דוגמה מעשית לשימוש בשער AOI כחלק מתכנון גדול יותר. בתרשים הבא מוצג MUX, כשליד כל רכיב מצוין כה טרנזיסטורים הוא מכיל לפי התרשימים שראינו בשבוע שעבר:



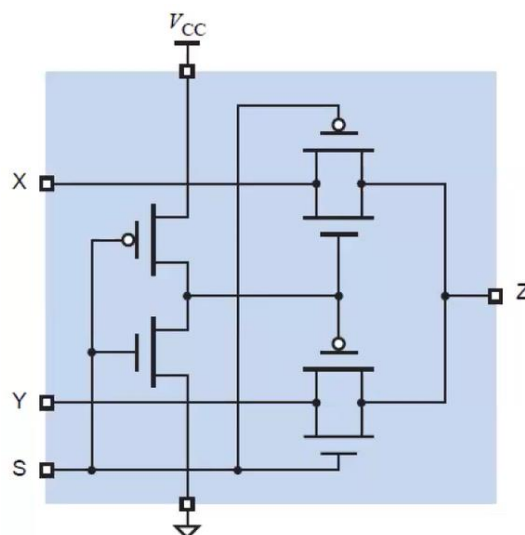
סך הכל יש צורך ב-20 טרנזיסטורים. אבל אם נשים לב, חלק גדול מהתכנון הוא שני שערי AND שנכנסים לשער OR, ואפשר להמיר אותם למבנה AOI עם מהפך אחריו. כך נוכל להשתמש ב-12 טרנזיסטורים: 2 למהפך השמאלי, 8 למבנה AOI שיחליף את שני שערי ה-AND שנכנסים לשער OR ונקודת 2 למהפך שנוסיף מימין, כדי לבטל את ההיפוך הלוגי שגורם מבנה AOI.

מימוש MUX באמצעות Transmission Gate

בתרשים הבא מוצג שער לוגי שנקרא Transmission Gate:



לטרנזיסטור התחתון נכנס האות הלוגי EN ולעליון נכנס ההופכי. אם EN הוא 1, הטרנזיסטור התחתון מקבל 1 ונפתח כי הוא n-channel והטרנזיסטור העליון מקבל את ההופכי – כלומר 0, ונפתח כי הוא p-channel. אם EN הוא 0, הטרנזיסטור התחתון מקבל 0 ונסגר כי הוא n-channel והטרנזיסטור העליון מקבל את ההופכי – כלומר 1, ונסגר כי הוא p-channel. הצורך בשני טרנזיסטורים הוא כדי שתמיד יהיה בין צמד הטרנזיסטורים שדה חשמלי. זה נצרך, כי במבנה זה אין מה שמאלץ את מתח ה-SUBSTRATE של הטרנזיסטורים ולכן המתח ב-GATES לא יכול ליצור שדה בפני עצמו כשאין מתח אחר מהכיוון השני. את ה-MUX שראינו לעיל, אפשר לממש באמצעות 6 טרנזיסטורים בלבד תוך שימוש ב-Transmission Gate, כפי שרואים בתרשים הבא:

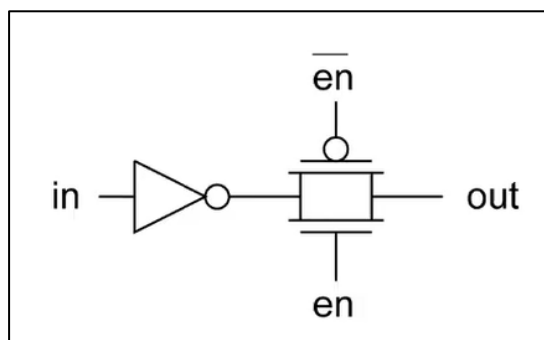


האות S (שמסמן select) נכנס לטרנזיסטור העליון של ה-Transmission Gate וגם לטרנזיסטור התחתון של ה-Transmission Gate. בנוסף, הוא נכנס לתוך מהפך, והמוצא של המהפך, שמכיל את ההופכי של S נכנס לטרנזיסטור העליון של ה-Transmission Gate וגם לטרנזיסטור התחתון של ה-Transmission Gate העליון.

כאשר S הוא 0, שני הטרנזיסטורים של ה-Transmission Gate העליון נפתחים – העליון כי מקבל את S שהוא 0 והתחתון כי מקבל את ההופכי שהוא 1. לעומת זאת, שני הטרנזיסטורים של ה-Transmission Gate התחתון נסגרים – העליון כי מקבל את ההופכי של S שהוא 1 והתחתון כי מקבל את S שהוא 0. לכן ביציאה יתקבל הערך של X. כאשר S הוא 1, יתרחש הכל להפך וביציאה יתקבל הערך של Y.

אמנם מבנה זה הוא חסכוני בטרנזיסטורים אך יש לו חסרון: בתכנונים שראינו עד עכשיו, היציאה מתחברת ישירות ל-VDD או ל-GROUND, אך במבנה הנוכחי היציאה מקבלת את האות תוך חיבור לאחת הכניסות דרך ה-channel של הטרנזיסטורים. זה גורם לזרם בטרנזיסטור ומחמם אותו. במצבים מסוימים החימום עלול להיות גדול מאד.

כמו שה-Transmission Gate מאפשר יצירת MUX, הוא מאפשר גם יצירה של מבנים לוגיים אחרים, כמו Tri-State Buffer (שהכרנו בשבוע 6), באופן הבא:

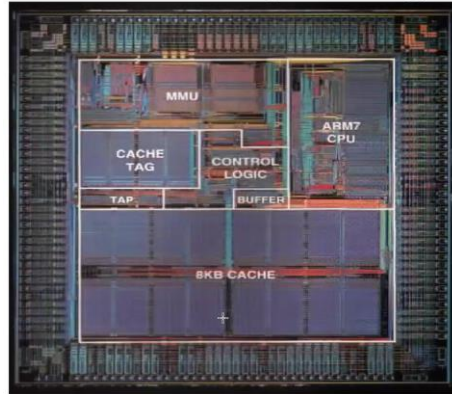


טכנולוגיות למימוש מעגלים גדולים על סיליקון – פתיחה

כדי ליצור מעגל ספרתי רציני, שכולל מאות מיליוני gates, ישנן כמה טכנולוגיות. במסגרת הקורס נכיר את העיקריות. חלק מהטכנולוגיות הן ייצור חד-פעמי, ובחלקן אפשר לממש שוב ושוב תכנונים שונים על אותה פיסת סיליקון.

טכנולוגיית Full Costume

בטכנולוגיה זו מייצרים בכל אזור בסיליקון טרנזיסטורים בהתאם לתפקידים השונים של האזורים. התמונה הבאה היא תמונה של רכיב כזה שנוצרה באמצעות מיקרוסקופ אלקטרוני ועוד כמה עזרים:

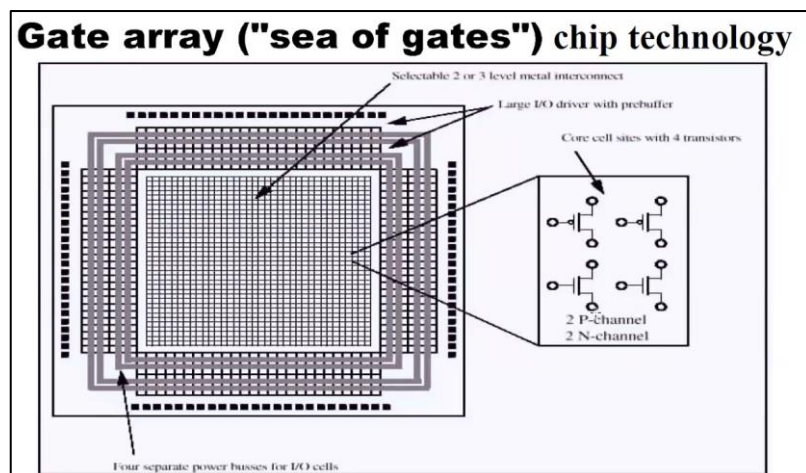


החלק הפנימי הוא ה-Core Logic, וחלק החיצוני, שמכיל את החיבורים לפינים מחוץ לאריזת הרכיב, נקרא I/O Ring.

היתרון של השיטה הוא חסכון במקום – כי הוא מתוכנן בדיוק לפי הצורך של התכנון הספציפי. החסרונות הם: עלות גבוהה וזמן ייצור ארוך בין סיום הסינתזה לבין המוצר המוגמר. כמו כן, טעויות בתכנון יעלו ביוקר כי אין דרך קצרה ומהירה לתקן אותן.

טכנולוגיית Gate Array

בטכנולוגיה זו, מכינים מראש בתוך פיסת סיליקון כמות עצומה של שערי NAND בעלי 2 כניסות, כפי שמופיע בתרשים הבא:



כפי שלמדנו, משערים אלה אפשר לבנות כל שער לוגי וממילא גם כל רכיב לוגי וכל Flip-Flop שנרצה (כי כל Flip-Flop בנוי משערים לוגיים). לכן, כל תכנון יכול להתממש על הרכיב בתנאי שנחבר נכון את שערי ה-NAND. בטכנולוגיה זו אפשר לייצר יותר מהר את המימוש הפיזי של התכנון, כי צריך לייצר רק את החיבורים (חיווט מוליכים) ולא לייצר את כל הטרנזיסטורים מאפס.

שבוע 10

המשך טכנולוגיית Gate Array

השכבה שכוללת את החיבורים ומחוברת למטריצת השערים נראת שכבת ה-METAL. אי אפשר להחליף את החיווט (שכבת ה-METAL) מרגע שהוא נעשה במפעל. גם כאן, טעות תגרום לצורך בייצור מחדש. כשרוצים לממש תכנון ב-Gate Array ישנו מדד שנקרא "Gate Count" שפירושו – כמות שערי ה-NAND שיהיה צריך בשביל לממש את התכנון. בטבלה זו מופיע כמה שערי NAND צורכים רכיבים נפוצים:

Element	Gate count
2-input nand	1
inverter	1
2-input and	2
2-input or	3
2-input nor	4
2-input xor	5
tri-state buffer	1 (many gate array architectures are not freely allowing the use of tri-state buffers in the design, other than "inout" ports of the entity)
Flip-Flop	10
Single RAM bit	2 (not including address decoder)
single ROM bit	1 (not including address decoder)

בטבלה הבאה מופיעה דוגמה למשפחה של רכיבים, עם כמות השערים שאפשר לנצל בכל אחד מהם:

Part Number	Usable Gates ¹		I/O Bond Pads
	2LM	3LM	
AM15HG9	4600	6100	72
AM15HG21	10300	13750	104
AM15HG39	19400	25800	148
AM15HG50	25000	33300	164
AM15HG83	41400	55200	212
AM15HG120	60000	80000	260
AM15HG186	93100	124000	324
AM15HG273	136750	182000	392
AM15HG377	188300	251000	460
AM15HG658	328900	438500	456
AM15HG901	450630	600000	532
AM15HG1139	569400	759000	600
AM15HG1404	700000	935000	664
AM15HG1683	841000	1121000	728
AM15HG1986	990000	1320000	792

בשני הטורים האמצעיים מופיעים כמות השערים שאפשר לנצל בכל רכיב בשתי אפשרויות שונות: משמאל מופיעה האפשרות של שימוש ב-2 שכבות METAL ומימין מופיעה האפשרות של שימוש ב-3 שכבות METAL. (2LM = 2 layers metal, 3LM = 3 layers metal). הסיבה שעם שכבה נוספת אפשר לנצל יותר שערים היא כי מתאפשר חיבור יותר מורכב של שערים באמצעות יותר 'גשרים' של חיבורים זה מעל זה. מבחינה כלכלית, לכל אחת משתי האפשרויות יש את היתרון שלה על פני האחרת:

- אם משתמשים בשתי שכבות, ייצור השכבות זול יותר, אבל צריך לשלם על יותר שטח סיליקון כדי לנצל מספיק שערים עבור מימוש התכנון.
- אם משתמשים בשלוש שכבות, ייצור השכבות יקר יותר, אבל צריך לשלם על פחות שטח סיליקון כדי לנצל את השערים הדרושים עבור מימוש התכנון.

הטור הימני ביותר בטבלה מציין כמה חיבורים לפינים של כניסות ויציאות מכיל כל רכיב במשפחה. האזור בתוך הציף שמכיל את החיבורים לפינים של הכניסות והיציאות נקרא I/O ring. לפעמים ניתקל במצב שבו יהיו ברכיב הקטן מספיק שערים לוגיים, אבל נצטרך לקחת רכיב גדול בגלל צורך ביותר כניסות ויציאות ממה שיש ברכיב הקטן. מצב כזה נקרא IO limited design. המצב ההפוך, שבו יהיו ברכיב הקטן מספיק כניסות ויציאות, אבל נצטרך לקחת רכיב גדול בגלל צורך ביותר שערים ממה שיש ברכיב הקטן, נקרא core limited design.

[הערה: כאשר מדברים על כמות שערים מקובל לציין אלף שערים באמצעות הביטוי k-gates כלומר kilo [gates]

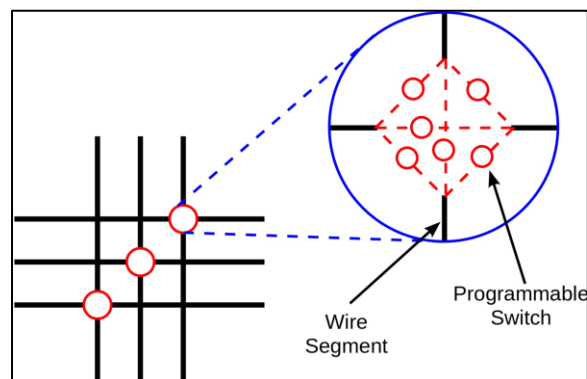
חסרונות שיטת Gate Array :

- מימוש של שערים פשוטים, שבתכנון ייעודי היו לוקחים מעט טרנזיסטורים, יקחו בשיטה זו הרבה טרנזיסטורים, כי הם ישתמשו בכמה שערי NAND כדי ליצור את הפונקציה הבוליאנית שלהם.
- אמנם זמן הייצור לא מאד ארוך, אבל הוא עדיין ארוך.

טכנולוגיית FPGA

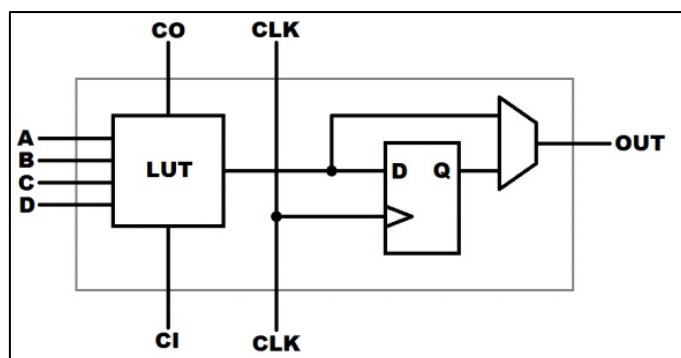
טכנולוגיית FPGA (Field Programmable Gate Array) מבוססת על הרעיון של Gate Array, אבל בשביל לחבר בין השערים הלוגיים שקיימים בסיליקון אין צורך במפעל שייצר Metal Layer, אלא אפשר לצרוב את החיבורים האלה באמצעות מחשב בבית. חיבורי המתכת קיימים כבר ברכיב, אבל יש אפשרות לחבר או לנתק ביניהם.

בשביל להבין איך הטכנולוגיה עובדת, נסתכל על התרשים הבא, שמייצג צומת של חיבורי מתכת :



כדי לחבר בין החוטים המגיעים לצומת, מוכנסים טרנזיסטורים שאפשר באמצעותם לחבר או לנתק, בדומה למה שלמדנו על השימוש בטרנזיסטורים למיתוג וחיבור מתחים. הטרנזיסטורים האלה מייקרים את הרכיב, כי הם מוסיפים עוד רכיבים, ובנוסף הם גם מוסיפים delay במעבר האות בחיווט שבין הרכיבים הלוגיים. אבל היתרון העצום של השיטה הוא היכולת לממש תכנון ברכיב פיזי במהירות גדולה בבי או במעבדה, וכן יכולת לתכנן שוב ושוב תכנונים שונים על אותו רכיב. כשהרכיב הזה מנותק מהמתח, הטרנזיסטורים כולם כבויים, ולכן בכל הדלקה של הרכיב יש לחבר שוב את החיבורים שבין החוטים. עם התקדמות הטכנולוגיה של ה-FPGA התחילו להשתמש במטריצה של רכיבים שאינם NAND אלא רכיב שנקרא Logic Cell (בקיצור LC) או Logic Element (בקיצור LE)⁷, ומוצג כאן בתרשים :

⁷ השם Logic Cell ניתן על ידי חברת XILINX. השם Logic Element ניתן על ידי חברת Altera.



החלק שנקרא LUT (Look Up Table) מכיל 'טבלת אמת' עם 4 כניסות ויציאה אחת. אפשר כמובן לחבר כמה כאלה אם יש צורך ביותר כניסות או יותר יציאות, באמצעות חיבור צמתים של חוטים, כפי שראינו. הרכיב הזה מכיל גם Flip-Flop, שאפשר להשתמש בו או לעקוף אותו, תלוי ב-MUX שנמצא אחרי ה-Flip-Flop. בכל הדלקה של הרכיב יש צורך למלא מחדש את טבלאות האמת של ה-LUTs וה-MUXים הקיימים ברכיבי ה-Logic Cell.

כל רכיב כזה יכול לשמש או כלוגיקה קומבינטורית (באמצעות ה-LUT) או כ-Flip-Flop. כך אפשר לקבוע מה עושה כל רכיב כזה, באמצעות הכנסת המתחים הנכונים ל-LUT ול-MUX, ולחבר בין הרכיבים שיצרנו באמצעות חיבור הצמתים על ידי טרנזיסטורים.

במשפחת רכיבי FPGA בשם Spartan-3 התחיל שימוש במטריצה של רכיבים גדולים יותר מ-Logic Cell ונקראים slices. כל אחד כזה כולל שני LUTs ושני FFs. בסדרות מתקדמות יותר יש שימוש ברכיבים גדולים אף יותר.

החיסרון של טכנולוגיית FPGA הוא שתמיד צריך לשלם על כל מה שנמצא בסיליקון, גם אם לא משתמשים בכולם.

בחירת רכיב FPGA עבור מימוש התכנון

כשבאים לממש תכנון בתוך רכיב FPGA, צריך להסתכל בדפי הנתונים של הרכיב, ולראות מה גודל התכנון שיכול להיות ממומש בו.

לפעמים היצרן נותן מספר שערים לוגיים שניתנים לשימוש ברכיבים. במקרים אלה, אם נחלק את המספר הזה ב-26 – שזו בערך כמות השערים הלוגיים הנצרכים ליצירת logic cell (16 עבור ה-LUT ו-10 עבור ה-Flip-Flop), נוכל לדעת כמה logic cells אפשר לממש ברכיב. אך זו הערכה מקורבת, שלא תמיד תשקף את המציאות.

המספר המדויק יותר שמופיע בדפי הנתונים של ה-FPGA הוא מספר ה-Logic Cells שיש ברכיב, שאותו יש להשוות למספר ה-Logic Cells שצריך התכנון הממומש.

נתון חשוב נוסף שנמצא בדפי הנתונים הוא גודל האריזות של הרכיבים השונים. פעמים רבות, רכיב מסוים יכול להתאים לכמה גדלים של אריזות.

בטבלה הבאה, הלקוחה מתוך דפי נתונים של משפחת הרכיבים Spartan-3, מופיעים הרכיבים במשפחה עם המשאבים שהם מכילים (שימו לב בעיקר לשני הטורים השמאליים, מספר ה-gates ומספר ה-cells):

Device	System Gates	Equivalent Logic Cells ¹	CLB Array (One CLB = Four Slices)			Distributed RAM Bits (K=1024)	Block RAM Bits (K=1024)	Dedicated Multipliers	DCMs	Maximum User I/O	Maximum Differential I/O Pairs
			Rows	Columns	Total CLBs						
XC3S50 ²	50K	1,728	16	12	192	12K	72K	4	2	124	56
XC3S200 ²	200K	4,320	24	20	480	30K	216K	12	4	173	76
XC3S400 ²	400K	8,064	32	28	896	56K	288K	16	4	264	116
XC3S1000 ^{2,3}	1M	17,280	48	40	1,920	120K	432K	24	4	391	175
XC3S1500 ⁴	1.5M	29,952	64	52	3,328	208K	576K	32	4	487	221
XC3S2000	2M	46,080	80	64	5,120	320K	720K	40	4	565	270
XC3S4000 ³	4M	62,208	96	72	6,912	432K	1,728K	96	4	712	312
XC3S5000	5M	74,880	104	80	8,320	520K	1,872K	104	4	784	344

בטבלה הבאה, הלקוחה מאותם דפי נתונים, מופיעים הרכיבים עם האריזות האפשריות עבור כל אחד:

Device	VQ100	CP132 ⁽¹⁾	TQ144 ⁽¹⁾	PQ208	FT256	FG320	FG456	FG676	FG900	FG1156
XC3S50	1	1.5	1.5	2	-	-	-	-	-	-
XC3S200	1	-	1.5	2	3	-	-	-	-	-
XC3S400	-	-	1.5	2	3	3	5	-	-	-
XC3S1000	-	-	-	-	3	3	5	5	-	-
XC3S1500	-	-	-	-	-	3	5	6	-	-
XC3S2000	-	-	-	-	-	-	5	6	9	-
XC3S4000	-	-	-	-	-	-	-	6	10	12
XC3S5000	-	-	-	-	-	-	-	-	10	12

שיעורי השלמה

[הסמסטר היה באורך 11 שבועות (במקום 13), בעקבות מלחמת חרבות ברזל. אק
נוסף לכך, שיצור אחד התבטל בעקבות הפחיתות לרשויות המקומיות. כהשלמה
לשיצור שהתבטל, עלו למודל הקלטות השלמה.]

מידע מעשי עבור שימוש בטכנולוגיית FPGA

שני היצרנים העיקריים של רכיבי FPGA הם חברת XILINX – שכיום היא חלק מחברת AMD וחברת Altera – שכיום היא חלק מ-Intel ונקראת Intel FPGA. אם ניכנס לאתר של XILINX, בדף שכתובתו www.xilinx.com/products/silicon-devices/fpga.html, נוכל לראות את המשפחות השונות של הרכיבים, מחולקות סדרות לפי הטכנולוגיה, מהחדשה לישנה. בסדרות המשפחות הישנות יותר, מצוין באתר עד מתי הם מתכוונים לייצר אותה (מידע חשוב עבור תכנון פיתוח וייצור לטווח הרחוק של שנים). כשבחרים משפחה מתוך אחת הסדרות, ייפתח דף עם הפרטים של הרכיבים באותה משפחה (IO pins, Logic Cells וכו'). כמו כן, מצוי שם מידע על הארזות האפשריות עבור כל משפחה, תוך ציון איזו אריזה מתאימה לאלו רכיבים מתוך המשפחה. כדי לממש תכנון על רכיב FPGA, יש צורך בתוכנת Place & Route שתוכל לצרוב את התכנון לתוך הרכיב (באמצעות הטענת מתחים נכונים לחיבורי הצמתים השונים). התוכנה של חברת XILINX המיועדת לכך נקראת Vivado ובסמסטר הבא נלמד לעבוד איתה. היא מתאימה רק לרכיבים של XILINX. אפשר להוריד את התוכנה מהאתר שלהם (במדור software). יש גרסה חינמית מצומצמת שאפשר להוריד למחשב (והיא תופסת לא מעט מקום במחשב), אך במחשבים של המעבדה שלנו יש את התוכנה המלאה. הדף הרלוונטי של החברה השנייה – Altera, שכולל מידע על רכיבי ה-FPGA שהם מייצרים, נמצא בכתובת www.intel.com/content/www/us/en/products/details/fpga.html. גם להם יש תוכנה משלהם עבור Place & Route ששמה Quartus והיא מתאימה דווקא לרכיבים של Altera.

טכנולוגיית Structured ASIC

אחרי שסיימנו את הטיפול הנרחב בטכנולוגיית FPGA, נחזור ללמוד עוד טכנולוגיות. חסרון בולט של טכנולוגיית Gate Array שהכרנו לעיל, הוא שכל תכנון יצטרך להסתמך אך ורק על שערי NAND, מה שעלול להפוך אותו למסורבל יותר מאשר תכנון מדויק וייעודי ב-Full Custom. בטכנולוגיית FPGA מסתמכים על Logic Cells, שהם רכיבים מפותחים יותר משערי NAND. טכנולוגיית Structured ASIC דומה ל-Gate Array, אך במקום להשתמש בשערי NAND היא משתמשת במטריצה של Logic Cells, שהחיווט ביניהם ייבנה פיזית (באופן חד-פעמי, שאינו ניתן למחיקה או שינוי) לפי התכנון המבוקש, כמו ב-Gate Array. בדומה ל-FPGA גם בטכנולוגיה זו יש רשימות של משפחות, ולכל משפחה יש ארזות אפשריות שונות לרכיבים של המשפחה.

טכנולוגיית CPLD

כדי להכיר את טכנולוגיית CPLD נקדים ונכיר את רכיבי PAL/GAL שמתוארים כאן:

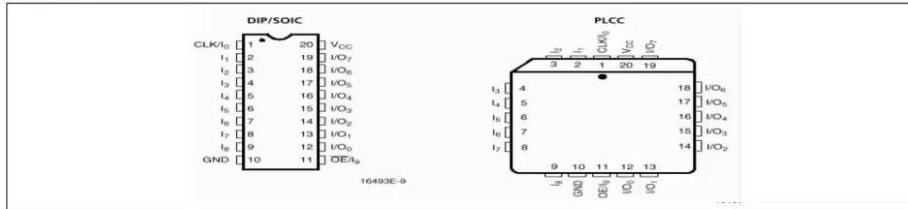
PAL/GAL devices

PAL	Programmable Array Logic	Trademark of AMD
GAL	Generic array Logic	Trademark of Lattice

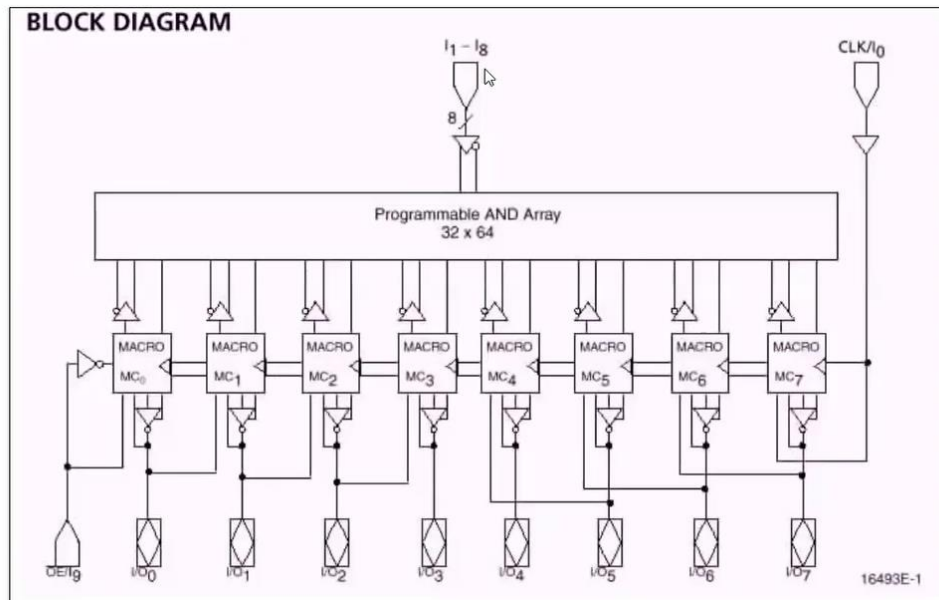
Small programmable logic device, first introduced in the mid 1980-s

The 16v8 device architecture

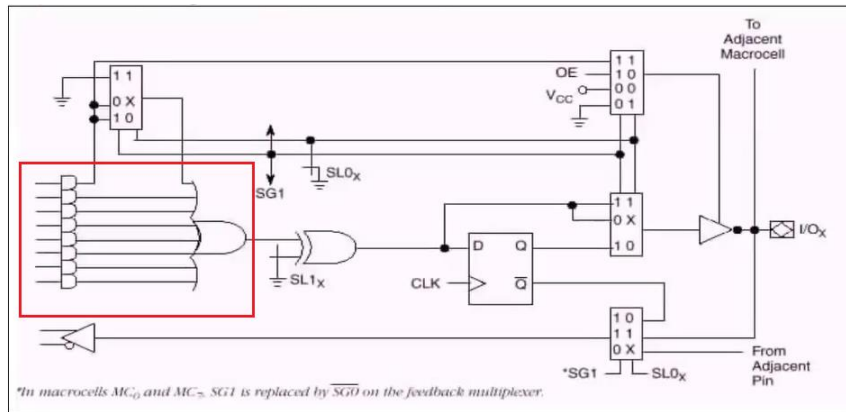
Pinout



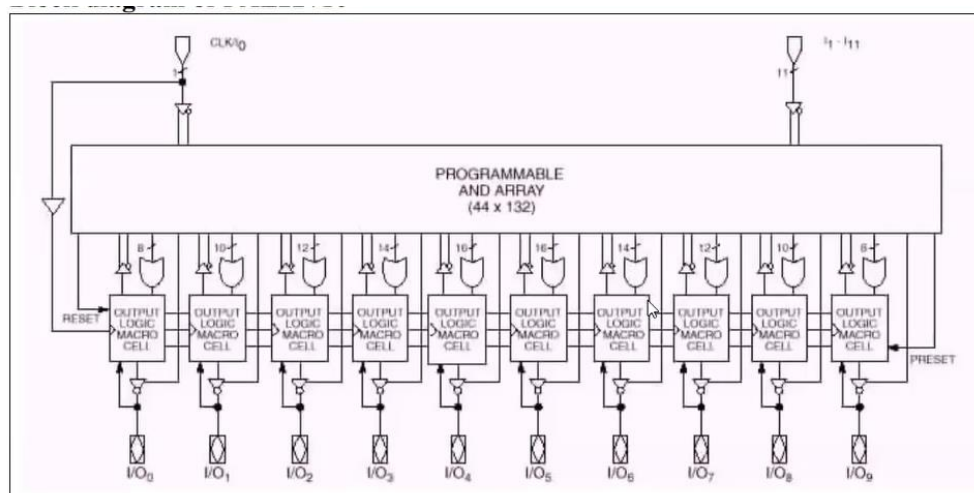
אלו רכיבים שאפשר לממש עליהם תכנונים שוב ושוב, שיוצרו על ידי שתי חברות שונות. הרכיבים מבוססים על ארכיטקטורת 16v8 שמוצגת בתרשים הבא :



הרכיב למעלה באמצע מציין 8 כניסות, למטה יש 8 I/Os ובאמצע קיימים 8 רכיבים שנקראים MACRO cells. מעל מערך ה-MACRO cells קיימת מטריצה של שערי AND שניתנת לתכנות, על מנת ליצור את החיבורים בין ה-MACRO cells. היחס בין 16 כניסות וואו יציאות לבין 8 רכיבי MACRO cells הוא שהעניק לארכיטקטורה את השם 16v8. המבנה הפנימי של ה-MACRO cells מתואר בתרשים הבא :

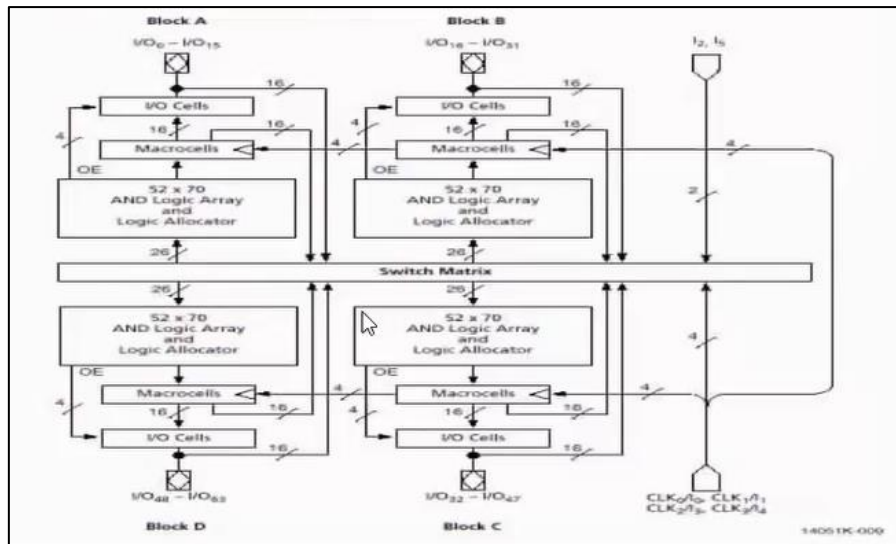


הסבר חלקי (ההסבר המלא בהקלטה): הרכיב מזכיר מאד את המבנה של Logic Cell שהכרנו בטכנולוגיית FPGA. החלק המוקף במרובע אדום אחראי ללוגיקה הקומבינטורית שאפשר לממש ב-MACRO cell. מדובר על שערי OR שהיציאות שלהם נכנסות לשער AND, מה שמאפשר מימוש של נוסחה בוליאנית שמגיעה בצורת POS (product of sums). היציאה של חלק זה נכנסת לשער XOR, שמתפקד כמעביר רגיל או כמהפך, תלוי בערך של הכניסה השנייה שלו – $SL1_x$. משער ה-XOR ממשיך האות ל-FF או עוקף אותו, ויכול גם לחזור אחורה, ולצאת דרך ה-buffer שנמצא למטה משמאל. בשונה מרכיבי FPGA, שבהם התכנון מתנדף עם כיבוי המתח⁸, ברכיבים אלו התכנון נשאר גם אחרי כיבוי המתח. בהתחלה היו מיוצרים רכיבים שניתנים לתכנות באופן חד-פעמי בלבד (OTP = one time programmable), אך בהמשך השתמשו ברכיבים שאפשר לממש עליהם תכנונים שונים שוב ושוב. בטכנולוגיה זו יש ארכיטקטורות נוספות אפשריות, כמו למשל 22v10, שכוללת 22 כניסות/יציאות ו-10 רכיבי MACRO cells, כמו שמוצג כאן:



ברכיבים אלו, כמו ברכיבי FPGA אין אפשרות לשימוש ביותר Flip-Flops ממה שמוגדר. אמנם תיאורטית אפשר 'לבנות' Flip-Flop נוסף באמצעות שערים לוגיים, שהרי Flip-Flop הוא חיבור של שערים לוגיים, אבל כלי הסינתזה לא ייתן את האופציה הזו. לכן, תכנון שמצריך יותר FFs יצטרך להיות ממומש על ארכיטקטורות יותר גדולות. טכנולוגיית CPLD (Complex Programmable Logic Devices) משתמשת במטריצה שכוללת הרבה בלוקים של רכיבי PAL/GAL, כמו שמומחש בתרשים הבא:

⁸ יוצא דופן אחד הוא רכיב MAX FPGA של Intel שבה התכנון אינו נדיף, ונשאר גם לאחר כיבוי המתח.

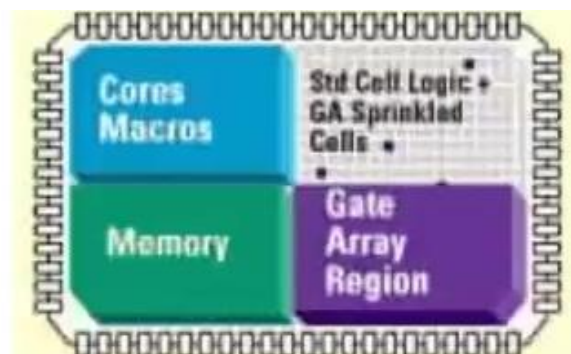


טכנולוגיית Standard Cell

רוב, המושג 'standard cell' מתייחס לספרייה שמיועדת עבור תכנון שימוש בטכנולוגיית Full Custom, שאותה הכרנו לעיל.

לפעמים המושג 'standard cell' מתייחס לטכנולוגיה שמשלבת בין Gate Array לבין Full Custom. לטכנולוגיה זו יש שם נוסף: Embedded Array. נלמד כאן על הטכנולוגיה הזו ואיך היא משלבת יתרונות של Gate Array ושל Full Custom.

אחד החסרונות של טכנולוגיית Gate Array הוא שיש מבנים לוגיים שאי אפשר לממש רק באמצעות שערי NAND, כמו למשל tri-state buffer (שהכרנו בשבוע 6). בנוסף, בניית זיכרונות בטכנולוגיית Gate Array לוקחת הרבה שערי NAND שלא יהיה אפשר לממש בהם לוגיקה קומבינטורית. חסרונות אלה לא קיימים בטכנולוגיית Full-Custom, אך הייצור שלה יותר איטי. כדי לנסות 'להנות משני העולמות', קמה טכנולוגיית standard cell, המיוצגת בתרשים הבא:



על אותו רכיב קיים אזור שימוש בשיטת Full Custom ואזור שימוש ב-Gate Array. כך אפשר להתחיל את מימוש חלק מהתכנון (הזיכרונות בעיקר) על החלק של הרכיב שמותאם ל-Full Custom ובמקביל לעבודה של המפעל על החלק הזה, אפשר לפתח את שאר התכנון – שימוש באמצעות Gate Array, ואותו לייצר במהירות יחסית לאחר סיום העבודה על פיתוח התכנון, כי הייצור של Gate Array מהיר יותר מזה של full custom.

בהצלחה לכולם בהמשך התואר ובהמשך הדרך!